



RÉPUBLIQUE DU BÉNIN
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ D'ABOMEY-CALAVI

INSTITUT DE FORMATION ET DE
RECHERCHE EN INFORMATIQUE

BP 526 Cotonou Tel : +229 21 14 19 88
<http://www.ifri-uac.net> Courriel : contact@ifri.uac.bj



MÉMOIRE

pour l'obtention du

Diplôme de Master en Informatique

Option : Sécurité Informatique

Présenté par :

Abdel-Faridh MERE NIMA

Sécurité des modèles d'intelligence artificielle dans les systèmes de maintien de l'ordre : vulnérabilités, menaces et méthodes de protection.

Sous la supervision :

Dr (MC) M. Abel KONNON

Membres du jury :

Nom et prénoms du président	Grade	Entité	Président
Nom et prénoms de l'examineur	Grade	Entité	Examineur
Nom et prénoms du rapporteur	Grade	Entité	Rapporteur

Année Académique : –

Sommaire

Dédicace	ii
Remerciements	iii
Résumé	iv
Abstract	v
List of Figures	vi
List of Tables	vii
Liste des Algorithmes	viii
Liste des acronymes	ix
Introduction	1
1 Cadre conceptuel et théorique	3
2 Méthodologie et Architecture	22
3 Expérimentation	36
4 Résultats et discussion	40
Bibliographie	46
Bibliographie	52
Conclusion	53
Bibliographie	54
Bibliographie	54
Table des matières	56

Dédicace

Mes dedicaces

Remerciements

Nos remerciements

Résumé

Resume en francais

Mots clés :,.....

Abstract

Resume en anglais

Key words:,.....

List of Figures

1.1	Architecture du Générative Adversarial Networks (GANs) [24]	6
1.2	Statistique des attaques et de défense des modèles de ML [23]	9
1.3	Attaque par inférence de l'appartenance [23]	18
1.4	Taxonomie des attaques d'extraction de modèle [23]	19
2.1	Architecture globale de sécurité à 5 zones	24
3.1	Architecture du Cas expérimental : comparaison entre le modèle Baseline et le modèle Secured.	37

List of Tables

3.1 Robustesse adversariale de ResNet50 standard (littérature) 38

Liste des Algorithmes

1	Configuration YAML du pipeline sécurisé	30
2	Classe <code>SecurityGate</code> Contrôle automatique de sécurité	31
3	Inverse	35

Liste des acronymes

acroglo :

A la fois dans les deux *Glossaire* : [acroglo](#)

IA :

Intelligence Artificielle

Introduction Générale

1. Contexte et justification

Ces dernières années, l'intelligence artificielle (IA) est devenue une composante essentielle de nombreuses technologies, bouleversant les pratiques dans des secteurs variés comme la santé, la finance, l'éducation, ou encore la sécurité publique. En particulier, les forces de l'ordre intègrent progressivement des solutions basées sur l'IA pour améliorer l'efficacité de la surveillance, de la détection et de la prévention des actes criminels. Ces solutions incluent notamment les systèmes de reconnaissance faciale, les caméras intelligentes, la prédiction criminelle automatisée (predictive policing), ainsi que les dispositifs d'analyse comportementale.

Toutefois, cette intégration croissante de l'IA dans des environnements aussi sensibles soulève de nombreux défis, notamment en matière de sécurité des modèles, de fiabilité des décisions algorithmiques, de respect des droits fondamentaux et de conformité éthique. Les systèmes IA utilisés par les forces de l'ordre sont exposés à des risques d'attaques adversariales, d'empoisonnement de données, d'extraction de modèle, et présentent des biais qui peuvent amplifier les discriminations dans les processus d'identification ou de ciblage.

Un rapport marquant publié par le Belfer Center de Harvard (*Attacking AI*, 2019) souligne que les domaines liés au maintien de l'ordre figurent parmi les plus vulnérables face aux attaques ciblant les modèles d'intelligence artificielle. Dans un contexte où l'erreur ou la manipulation d'un algorithme peut conduire à une arrestation injustifiée ou à une violation des droits fondamentaux, il devient indispensable d'évaluer les vulnérabilités de ces modèles et de proposer des stratégies concrètes de sécurisation.

C'est dans cette optique que s'inscrit ce travail, avec pour ambition d'analyser les risques pesant sur les modèles IA déployés dans les systèmes de sécurité publique, et de proposer une méthode de sécurisation alliant défense technique, cadre éthique et approche DevSecOps.

2. Problématique

La multiplication des systèmes d'IA dans les forces de l'ordre s'est faite souvent de manière expérimentale ou commerciale, sans que des garanties suffisantes soient apportées sur leur robustesse, leur fiabilité et leur alignement aux principes d'équité, de transparence et de non-discrimination. En l'absence de sécurisation renforcée, ces systèmes sont exposés à de nombreuses menaces. Face à ces défis

de sécurité, il devient urgent de définir une méthode de sécurisation systématique des modèles IA utilisés dans les systèmes de maintien de l'ordre. Cette méthode doit couvrir l'ensemble du cycle de vie du modèle, intégrer des protections techniques, s'aligner sur les normes réglementaires (RGPD, AI Act, ISO 42001, Loi n°2017-20 au Bénin) et garantir un haut niveau d'éthique algorithmique.

3. Objectifs de l'étude

L'objectif principal de notre étude est de proposer une méthode globale de sécurisation des modèles d'intelligence artificielle utilisés dans les systèmes de maintien de l'ordre, en intégrant les aspects techniques, réglementaires et éthiques.

La réalisation de cet objectif permettra d'atteindre les objectifs spécifiques que sont :

- Identifier les principales vulnérabilités des modèles IA utilisés dans les forces de l'ordre.
- Analyser les attaques potentielles (empoisonnement, adversarial, extraction, biais).
- Mettre en œuvre une méthode de protection combinant adversarial training, chiffrement homomorphe, differential privacy, et DevSecOps.
- Proposer une preuve de concept basée sur un modèle open source (MobileNetV2) en contexte de sécurité publique.
- Formuler des recommandations éthiques et réglementaires pour les systèmes IA en contexte sécuritaire.

4. Organisation du mémoire

Ce mémoire est structuré en quatre chapitres principaux. Le premier chapitre présente les fondements théoriques de l'intelligence artificielle, les types de modèles utilisés, les cas d'usage spécifiques dans les systèmes de maintien de l'ordre, ainsi que l'analyse des vulnérabilités et menaces pesant sur ces modèles, en abordant les attaques adversariales, les biais algorithmiques, et les atteintes à la confidentialité. Le deuxième chapitre est consacré à la présentation de notre méthodologie de travail et de notre proposition de méthode globale de sécurisation des modèles IA, intégrant des techniques avancées ainsi qu'une approche DevSecOps pour la protection du cycle de vie des modèles. Le troisième chapitre décrit une application pratique de cette méthode à travers une expérimentation sur un modèle de reconnaissance d'objet dangereux. Enfin le quatrième chapitre présente une analyse des résultats obtenus issus de cette expérimentation.

Cadre conceptuel et théorique

Introduction

1.1 Introduction aux modèles d'intelligence artificielle

1.1.1 Notion d'intelligence artificielle

L'intelligence artificielle est un domaine de l'informatique qui vise à donner à des systèmes la capacité de décision et d'anticipation, en exécutant des tâches qui auraient nécessité l'intelligence humaine. [30] définit l'intelligence artificielle comme un algorithme autonome et capable de prendre des décisions et/ou de faire des prédictions en simulant l'intelligence humaine. Il s'agit des algorithmes ayant des capacités d'adaptation, de réflexion et d'apprentissage. C'est grâce à l'apprentissage qu'un système intelligent capable d'exécuter une tâche peut améliorer ses performances avec l'expérience. L'intelligence artificielle vise alors à reproduire au mieux, à l'aide de machines, des activités mentales, qu'elles soient de l'ordre de la compréhension, de la perception ou de la décision [30]. Les techniques d'intelligence artificielle nous permettent d'automatiser l'extraction de connaissances à des fins prédictives pouvant être utilisées pour prendre de meilleures décisions. Il existe deux types d'IA : l'**IA faible** ou "de niveau 1" qui se concentre sur une tâche spécifique, comme la reconnaissance vocale, et l'**IA forte** ou "de niveau 2" qui vise à simuler l'intelligence humaine dans sa globalité [6].

En intelligence artificielle (IA), un modèle d'Intelligence Artificielle est essentiellement un cadre ou un système conçu pour interpréter et apprendre à partir de données, simulant un niveau de compréhension et de prise de décision semblable à celui d'un être humain [7]. Il s'agit d'une application spécifique de technologies d'IA pour résoudre un problème particulier. Les modèles d'IA sont donc les implémentations concrètes de ces méthodes.

1.1.2 Les différents types de modèles d'IA

Il existe une grande variété d'approches pour concevoir et mettre en uvre un système d'intelligence artificielle. Le choix de l'approche détermine en grande partie le type de modèle d'IA qui sera utilisé pour résoudre un problème donné. Selon l'approche adoptée, le modèle d'IA peut varier en termes de structure, de mode d'apprentissage et de finalité. Comme le souligne [6, 7], ces différentes approches permettent de distinguer plusieurs types de modèles d'IA, chacun adapté à des tâches spécifiques

et présentant des avantages et des limites propres. Dans cette section, nous présentons les différents types d'apprentissage en IA ainsi que les principales architectures utilisées pour modéliser des systèmes intelligents.

1.1.2.1 Types d'apprentissage en intelligence artificielle

Les modèles d'IA peuvent être classés en trois grandes catégories d'apprentissage : l'apprentissage supervisé, l'apprentissage non supervisé et l'apprentissage par renforcement.

- **Apprentissage supervisé**

L'apprentissage supervisé repose sur l'utilisation d'un ensemble de données labellisées où chaque échantillon est associé à une sortie attendue. Le modèle apprend à mapper les entrées aux sorties correctes en minimisant une fonction de perte [18]. Parmi les algorithmes les plus courants, on trouve :

- **Les régressions linéaires et logistiques** : Utilisée pour les problèmes de régression afin de prédire une valeur basée sur des relations linéaires entre les variables.
- **Les machines à vecteurs de support (SVM)** : Utilisées pour des problèmes de classification en traçant une frontière optimale entre différentes classes dans les données.
- **Arbres de décision (Decision Trees)** : Modèles visuels qui décomposent les données en étapes successives pour arriver à une prédiction. Ils sont simples à comprendre et interpréter.
- **Forêts aléatoires (Random Forests)** : Une extension des arbres de décision, combinant plusieurs arbres pour réduire le risque d'erreurs et améliorer la précision.

- **Apprentissage non supervisé**

Contrairement à l'apprentissage supervisé, l'apprentissage non supervisé utilise des données non étiquetées. L'objectif est de trouver des structures ou des patterns cachés dans les données [18]. Les principales méthodes incluent :

- **Le clustering**, avec des algorithmes tels que K-Means ou DBSCAN, qui regroupent les données en fonction de leur similarité.
- **Les modèles de réduction de dimensionnalité**, comme l'analyse en composantes principales (PCA) utilisée pour réduire la dimensionnalité des données tout en conservant le maximum d'informations.
- **Les auto-encodeurs**, des réseaux neuronaux capables d'apprendre une représentation des données en compressant et en reconstruisant leurs représentations.
- **Association Rules (Règles d'association)** : Identifie les relations fréquentes entre les variables dans un ensemble de données, souvent utilisées dans l'analyse des paniers d'achat.

- **Apprentissage par renforcement**

Dans l'apprentissage par renforcement, un agent apprend à interagir avec un environnement en maximisant une fonction de récompense. Cette approche est particulièrement efficace pour des problèmes complexes comme les jeux ou la robotique [18]. Les algorithmes clés comprennent :

- **Q-Learning** : Un algorithme basé sur une table qui met à jour les valeurs Q pour chaque combinaison état-action, permettant à l'agent d'apprendre une politique optimale au fil du temps.
- **Deep Q-Networks (DQN)** : Extension du Q-Learning utilisant des réseaux neuronaux pour approximer les valeurs Q, particulièrement utile pour des environnements complexes avec un grand nombre d'états.
- **Politique de Monte Carlo** : Approche où l'agent apprend à partir d'épisodes complets de ses interactions avec l'environnement, en calculant les récompenses cumulées pour chaque épisode.
- **Algorithme Actor-Critic** : Combinaison de deux réseaux neuronaux : un acteur pour choisir les actions et un critique pour évaluer celles-ci, améliorant la convergence et l'efficacité de l'apprentissage.
- **Algorithmes de Gradient de Politique (Policy Gradient)** : Ces modèles apprennent directement une politique en optimisant les récompenses attendues grâce à des techniques de gradient.

1.1.2.2 Architectures avancées des modèles d'IA

Avec l'augmentation des besoins en modélisation, plusieurs architectures avancées ont été développées pour traiter des problèmes plus complexes.

- **Réseaux neuronaux convolutifs (CNN)**

Les réseaux neuronaux convolutifs (CNN, pour *Convolutional Neural Networks*) sont spécialement conçus pour le traitement des images et utilisent des couches de convolution pour extraire des caractéristiques essentielles. [18] précise que ces architectures sont à la base des systèmes modernes de reconnaissance faciale et de vision par ordinateur, l'analyse vidéo.

- **Réseaux neuronaux récurrents (RNN)**

Les réseaux neuronaux récurrents (RNN, pour *Recurrent Neural Networks*) sont une classe de réseaux de neurones spécialement conçue pour traiter des données séquentielles ou temporelles. Leur structure unique leur permet de mémoriser des informations provenant des données précédentes grâce à des **boucles internes**. Cela leur confère la capacité d'analyser les relations entre les éléments des données dans un ordre séquentiel. Leur application les plus courantes sont : l'analyse des séries temporelles, le traitement du langage naturel, la reconnaissance vocale et transcription.

- **Transformers** Les Transformers ont révolutionné le domaine du traitement du langage naturel avec des modèles comme **BERT** et **GPT**. Leur mécanisme d'auto-attention permet de traiter les dépendances à longue distance de manière plus efficace que les RNN [25].

- **Général Adversarial Networks (GANs)**

Le Général Adversarial Networks (GANs) un type de modèle génératif composé de deux réseaux neuronaux en compétition : le **générateur (G)** qui crée des données artificielles (ex.

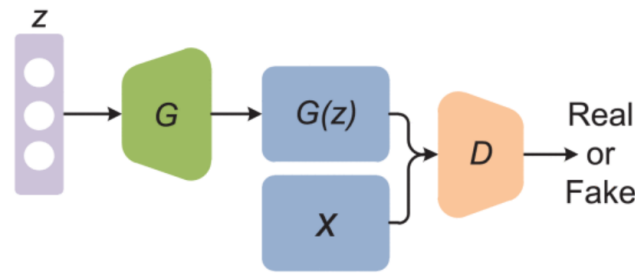


FIGURE 1.1 : Architecture du Générative Adversarial Networks (GANs) [24]

images) pour imiter la distribution des données réelles et le **discriminateur (D)** qui évalue si les données proviennent du générateur ou des données réelles. D'après [24] le but est un jeu à somme nulle (inspiré de la théorie des jeux) : le générateur essaie de tromper le discriminateur, tandis que ce dernier tente de distinguer le vrai du faux. L'objectif est d'atteindre un équilibre où le générateur produit des données presque indistinguables des données réelles.

Parmi les différents types de modèles, le choix du modèle à construire s'opère en fonction du besoin auquel doit répondre le système d'AI.

1.1.3 Importance et impact de l'IA dans les systèmes modernes

De nos jours, l'IA est omniprésente et trouve son application dans divers domaines de la vie. Parmi ces nombreux secteurs applications, on distingue :

- **La Médecine**

Avec les avancées fulgurantes de ce secteur, nous avons des systèmes d'assistant au diagnostic médical alimentés par des IA en analysant des images radiologiques ou en prédisant des maladies à partir de dossiers médicaux électroniques [13, 26].

- **Automobile**

L'IA occupe une grande place ici avec des véhicules autonomes qui utilisent des algorithmes d'IA pour percevoir leur environnement, prendre des décisions de conduite et naviguer en toute sécurité.

- **Finance**

Les institutions financières appliquent des modèles d'IA pour détecter des fraudes, évaluer des risques de crédit et optimiser des portefeuilles d'investissement [19].

- **Service client**

Les chatbots et les assistants virtuels, alimentés par des modèles de traitement du langage naturel, améliorent l'expérience client en fournissant des réponses rapides et précises.

- **Agriculture**

[19] nous indique que les techniques d'interprétation permettent de mieux surveiller les cultures et les ressources nécessaires, comme l'eau et les engrais, en analysant les données obtenues sur le terrain.

Avec cette croissante importance de l'IA dans des aspects critiques de la société, des préoccupations concernant la sécurité, la confidentialité et l'éthique ont émergé. Les modèles d'IA peuvent être vulnérables à des attaques adversariales, où de petites perturbations intentionnelles des données d'entrée peuvent induire des erreurs significatives [Szegedy et al., 2014]. De plus, des biais présents dans les données d'entraînement peuvent conduire à des décisions discriminatoires, soulevant des questions éthiques sur l'équité et la justice [Doshi-Velez & Kim, 2017]. Il est donc essentiel de développer des cadres robustes pour assurer la sécurité et l'éthique des systèmes d'IA.

1.2 Sécurité et intelligence artificielle

L'essor des systèmes d'intelligence artificielle dans des domaines critiques comme la sécurité publique, la justice prédictive, ou encore la surveillance soulève des enjeux majeurs en matière de sécurité informatique. Contrairement aux systèmes traditionnels, les modèles IA ne sont pas uniquement vulnérables au niveau du code ou de l'infrastructure, mais aussi dans leur logique d'apprentissage et de généralisation. Cette section introduit les principaux concepts de sécurité liés à l'IA et les vulnérabilités spécifiques à ces modèles.

1.2.1 Définitions et concepts clés

L'application des principes classiques de cybersécurité aux modèles IA repose principalement sur le triptyque **CIA(Confidentialité, Intégrité, Disponibilité)** pour garantir leur fiabilité et leur sécurité.

- **Confidentialité :**

La confidentialité concerne la protection des données d'entraînement et des modèles contre l'accès non autorisé. Les modèles d'IA traitent souvent des données sensibles (dossiers médicaux, visage des individus, transactions bancaires, etc.), ce qui les rend vulnérables aux violations de données. Des attaques comme **l'extraction de modèles** permettent aux attaquants de reconstruire un modèle en interrogeant son API, exposant ainsi des informations confidentielles.

- **Intégrité**

L'intégrité vise à empêcher toute modification non autorisée du modèle ou de ses données. Les **attaques adversariales** exploitent cette faiblesse en manipulant légèrement les entrées d'un modèle pour induire des erreurs dans ses prédictions. Par exemple, un algorithme de reconnaissance d'image peut être trompé par des modifications imperceptibles sur une image, entraînant des classifications erronées.

- **Disponibilité**

La disponibilité garantit que les modèles IA restent accessibles et opérationnels même en cas d'attaques ou de pannes. Des attaques **de déni de service (DoS)** ciblent les infrastructures IA en surchargeant les ressources, ralentissant ou interrompant leur fonctionnement. La **Sécurité informatique des modèles IA** intègre la surveillance de l'ensemble du pipeline d'apprentissage automatique, de l'acquisition des données jusqu'au déploiement du modèle.

1.2.2 Les menaces et vulnérabilités liées aux modèles d'IA

Les modèles peuvent être manipulés, trompés ou détournés à des fins malveillantes, même après leur déploiement. Ces failles peuvent compromettre la confidentialité des données, introduire des biais discriminatoires, ou engendrer des prises de décision erronées dans des systèmes critiques. On distingue généralement deux grandes catégories de vulnérabilités :

- **Les vulnérabilités structurelles ou algorithmiques** : liées à la conception même du modèle, à son architecture, à la qualité de l'apprentissage ou à la mauvaise gestion des biais.
- **Les vulnérabilités adversariales** : résultant d'attaques ciblées visant à tromper, extraire ou détourner le modèle.

Parmi les menaces les plus courantes, on peut citer :

- **Les attaques adversariales** : des perturbations invisibles mais efficaces ajoutées aux données d'entrée pour induire en erreur le modèle. Elles peuvent compromettre des systèmes critiques comme la reconnaissance faciale, la détection d'objets ou l'identification d'infractions dans des contextes de sécurité publique.
- **Les biais algorithmiques** : erreurs systématiques induites par des données non représentatives ou des choix de conception. Dans des applications comme la surveillance ou l'analyse comportementale, cela peut mener à des discriminations basées sur le genre, l'origine ethnique ou l'âge, remettant en cause l'équité des systèmes IA.
- **Le détournement ou vol de modèle** : reproduction illégale d'un modèle par extraction via API (attaques de type model stealing), ce qui met en péril la propriété intellectuelle et la confidentialité des données utilisées à l'entraînement. Ce type de vulnérabilité est particulièrement problématique dans les systèmes utilisés par les forces de l'ordre où l'accès aux modèles doit être strictement contrôlé.

1.2.2.1 Illustration de l'évolution des menaces et des défenses

Pour mieux comprendre l'évolution de ces menaces, la **Figure 1.2** issue du travail de [23] offre une vue d'ensemble précieuse.

La figure montre une **augmentation significative du nombre de publications scientifiques** portant sur les attaques contre les modèles d'apprentissage automatique, en particulier à partir de 2016. Si les publications sur les attaques dominaient au début, les travaux sur les **mécanismes de défense** ont connu une croissance notable dès 2018.

On y distingue notamment :

- La **prévalence des attaques par entraînement de modèle substitutif**, qui consistent à interroger un modèle cible pour entraîner un clone.
- L'émergence de **techniques de défense** :
 - **Proactives** : comme la perturbation de sortie (output obfuscation).
 - **Réactives** : comme la surveillance du comportement du modèle pour détecter des tentatives d'extraction ou d'empoisonnement.

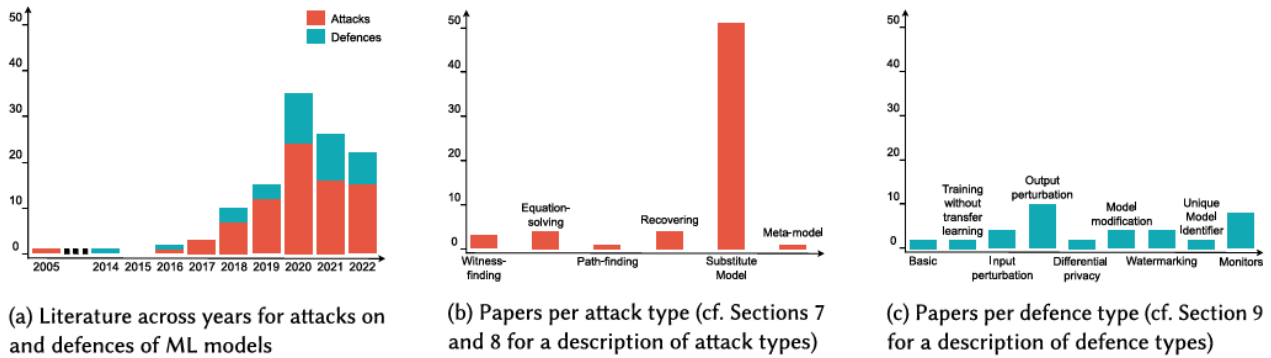


FIGURE 1.2 : Statistique des attaques et de défense des modèles de ML [23]

Cette progression illustre l'importance croissante accordée à la **sécurité spécifique des modèles IA** dans la communauté scientifique, et souligne la nécessité de proposer des approches robustes, intégrées et auditées.

1.3 Usages des modèles d'IA dans les systèmes de maintien de l'ordre

1.3.1 Contexte de l'intégration de l'IA dans la sécurité publique

L'intelligence artificielle transforme rapidement les outils et pratiques du maintien de l'ordre. De plus en plus de forces de police s'équipent de systèmes algorithmiques destinés à renforcer la surveillance, à anticiper les crimes ou à identifier rapidement les individus recherchés. Ces systèmes reposent souvent sur des modèles d'apprentissage profond (deep learning) adaptés à des tâches de classification, détection ou reconnaissance, intégrés dans des dispositifs intelligents.

Selon Brundage et al. (2018), les institutions publiques figurent parmi les secteurs les plus exposés à des abus algorithmiques en raison de la sensibilité des données traitées et de l'impact direct sur les libertés individuelles [10].

1.3.2 Principaux cas d'usage dans les forces de l'ordre

Les cas d'usage les plus répandus sont les suivants :

- **Reconnaissance faciale automatisée**

Les systèmes de reconnaissance faciale s'appuient sur des modèles de réseaux de neurones convolutifs (CNN) comme ResNet ou FaceNet. Ils sont utilisés pour comparer des visages capturés par caméras de surveillance à des bases de données judiciaires. De tels systèmes sont très souvent déployés dans des contextes variés : aéroports, centres urbains, contrôles d'identité.

Exemple : Clearview AI propose une base de données contenant plus de 3 milliards d'images extraites d'Internet, utilisée par plusieurs forces de l'ordre aux États-Unis [17].

- **Analyse comportementale via caméras intelligentes**

Des modèles IA embarqués dans des systèmes de vidéosurveillance permettent de détecter :

- des comportements suspects (mouvements anormaux, bagarres),

- des objets dangereux (armes, objets abandonnés),
- des intrusions dans des zones interdites.

Cette approche repose sur des techniques de vision par ordinateur en temps réel, couplées à des alertes automatiques.

- **Lecture automatique des plaques dimmatriculation (LAPI/ANPR)**

Les systèmes de **Lecture Automatique de Plaques dImmatriculation (LAPI)** ou **ANPR (Automatic Number Plate Recognition)** sont aujourd'hui largement déployés dans les forces de l'ordre, notamment pour :

- surveiller les flux routiers en temps réel,
- identifier des véhicules suspects ou volés,
- ou encore contrôler automatiquement les accès à des zones sécurisées.

Ces systèmes reposent sur des **réseaux neuronaux convolutifs (CNN)** entraînés pour détecter les véhicules, localiser la plaque, et lire les caractères via des techniques de reconnaissance optique de caractères (OCR) embarquées.

Ils peuvent fonctionner à partir de caméras fixes (péages, entrées de villes) ou mobiles (véhicules de patrouille) et sont souvent intégrés dans des solutions de vidéosurveillance plus larges.

- **Prédiction de la criminalité (Predictive Policing)**

Des outils comme **PredPol** utilisent des données historiques de crimes (lieux, dates, types) pour prédire les zones et périodes à risque. Ces algorithmes utilisent principalement des modèles d'apprentissage supervisé (SVM, régressions logistiques, réseaux de neurones).

Cependant, plusieurs études ont montré que ces systèmes peuvent **renforcer les biais historiques**, en concentrant les patrouilles dans des quartiers déjà sur-surveillés [20].

- **Vérification d'identité biométrique**

Des systèmes de contrôle d'accès ou de filtrage aux frontières utilisent des modèles IA pour la vérification biométrique : empreintes digitales, analyse vocale, iris, etc.

1.3.3 Risques et limites observés

Le rapport *Attacking AI* du Belfer Center (2019) identifie plusieurs failles potentielles dans ces usages :

- **Vulnérabilités adversariales** : des individus peuvent tromper les systèmes de reconnaissance faciale à l'aide de lunettes modifiées ou de patches imprimés [27].
- **Risques de surveillance de masse** : l'absence de régulation peut conduire à une surveillance intrusive de populations entières.
- **Atteintes à la vie privée** : collecte de données biométriques sans consentement.
- **Discriminations algorithmiques** : certains modèles présentent des taux d'erreurs plus élevés sur des groupes ethniques minoritaires [11].

1.3.4 Nécessité de régulation et d'encadrement

L'usage de ces modèles IA par les forces de l'ordre nécessite des **garanties techniques, éthiques et juridiques**, afin de prévenir les abus et de protéger les droits fondamentaux. Plusieurs textes normatifs (RGPD, AI Act) insistent désormais sur la nécessité d'auditer, expliquer et justifier les décisions algorithmiques, en particulier dans des domaines à haut risque comme la sécurité publique.

1.4 Éthique et cadre réglementaire des modèles IA

L'intégration croissante de l'intelligence artificielle dans des domaines sensibles tels que la santé, la finance et la justice soulève d'importants défis éthiques et réglementaires. L'usage des modèles IA doit être encadré par des principes de transparence, d'équité et de responsabilité, afin d'éviter les discriminations algorithmiques et les atteintes à la vie privée. De plus, plusieurs cadres réglementaires, tels que le **RGPD**, l'**EU AI Act** et la **norme ISO 42001**, ont été développés pour garantir une gouvernance responsable des systèmes d'IA. Cette section examine les principes fondamentaux de l'éthique en IA ainsi que les réglementations en vigueur.

1.4.1 Principes fondamentaux de l'éthique en IA

- **Transparence et explicabilité des modèles :**

La transparence est un élément central de l'éthique des modèles IA. Elle implique la capacité des utilisateurs et des régulateurs à comprendre comment un modèle prend ses décisions. Selon [19], un manque de transparence peut entraîner un manque de confiance et des erreurs aux conséquences graves, notamment dans des secteurs critiques comme la santé ou la justice.

- **Explicabilité des modèles :** Certains modèles d'apprentissage automatique, comme les réseaux neuronaux profonds, sont souvent qualifiés de boîtes noires en raison de leur complexité. Des techniques comme **LIME** (Local Interpretable Model-agnostic Explanations) et **SHAP** (Shapley Additive Explanations) permettent d'améliorer l'interprétabilité des modèles en identifiant les caractéristiques ayant influencé une décision [19, 26].
- **Traçabilité des décisions IA :** Dans certains secteurs comme la finance, il est essentiel de pouvoir expliquer pourquoi un crédit a été refusé ou accordé afin de respecter les exigences légales et éthiques.

- **Justice algorithmique et réduction des biais**

L'équité algorithmique vise à garantir que les modèles IA prennent des décisions justes et non discriminatoires. Toutefois, les biais dans les données d'entraînement ou dans la conception des modèles peuvent conduire à des résultats inéquitables.

1.4.2 Régulations et normes

L'intégration de l'intelligence artificielle dans les services de sécurité et de maintien de l'ordre impose une conformité stricte à des réglementations nationales et internationales. Ces cadres visent à prévenir les abus, protéger les libertés individuelles, et encadrer l'usage de technologies potentiellement intrusives ou discriminantes.

- **RGPD (Règlement Général sur la Protection des Données) :**

Entré en vigueur en mai 2018 dans l'Union européenne, le **RGPD** (Règlement UE 2016/679) impose un cadre juridique strict au traitement des données à caractère personnel, y compris biométriques. Il accorde aux individus un droit à la transparence, à la rectification, à l'effacement et à l'explication des décisions automatisées [8]. Dans le contexte sécuritaire, toute application de reconnaissance faciale ou d'analyse comportementale doit respecter ces obligations. Le traitement de données sensibles comme les empreintes faciales doit reposer sur une base légale solide et être proportionné aux objectifs poursuivis.

- **Loi n°2017-20 du Bénin sur la protection des données personnelles**

Le Bénin s'est également doté d'un cadre juridique national avec la **Loi n°2017-20 du 20 avril 2018** [1], relative à la protection des données à caractère personnel. Cette loi crée l'Autorité de Protection des Données Personnelles (**APDP**), chargée de veiller à la conformité des traitements. Les systèmes IA déployés par les forces de sécurité doivent être déclarés à l'APDP, et les citoyens doivent être informés des traitements automatisés dont ils peuvent faire l'objet.

- **AI Act de l'Union européenne (proposé en 2021)**

L'**AI Act** (Règlement européen sur l'IA) [4] est le premier texte de loi à classer les systèmes d'IA par niveaux de risques. Les systèmes utilisés dans les domaines de la sécurité publique, comme la surveillance biométrique en temps réel ou la reconnaissance faciale, sont classés à **haut risque**, voire **interdits** dans certains cas (article 5).

Ce texte impose des obligations strictes de transparence, d'explicabilité, de robustesse, de supervision humaine et de gestion des biais. Il prévoit aussi des audits réguliers et la documentation complète des systèmes.

- **Norme ISO/IEC 42001 :2023 sur les systèmes de management de l'IA**

Cette norme internationale, publiée en décembre 2023, fournit un cadre pour établir, mettre en œuvre et améliorer un système de gestion des technologies IA dans les organisations [3]. Elle inclut des directives sur :

- la gestion des risques liés à l'IA,
- l'éthique,
- la transparence des décisions algorithmiques,
- la responsabilité des développeurs et des utilisateurs.

Dans un environnement de sécurité publique, cette norme peut servir de base à une gouvernance rigoureuse des systèmes IA déployés par l'État.

1.5 Typologie des vulnérabilités des modèles d'IA

Les modèles d'intelligence artificielle, en particulier les réseaux neuronaux profonds, présentent des points faibles inhérents à leur conception, à la qualité des données utilisées pour l'entraînement, et aux conditions d'exploitation. Cette section propose une typologie des principales vulnérabilités exploitées dans les attaques contre ces systèmes. L'essor des modèles d'intelligence artificielle a révélé diverses vulnérabilités exploitées par des attaquants pour compromettre leur fiabilité et leur sécurité. Ces vulnérabilités peuvent être classées en plusieurs catégories :

1.5.1 Vulnérabilités structurelles et inhérentes aux modèles IA

Les modèles d'IA sont sensibles à des failles inhérentes à leur conception et à leur entraînement :

- **Sur-apprentissage (Overfitting)** : Lorsque le modèle s'adapte trop aux données d'entraînement, il devient inefficace face à de nouvelles données. Le modèle n'extraît pas les règles implicites mais apprend uniquement par cœur les données d'entraînement. Il est à nouveau incapable de généraliser efficacement ses prédictions à de nouvelles entrées [2].
- **Sensibilité aux données bruitées** : Les modèles peuvent être facilement trompés par des données bruitées ou adversariales [15]. Cela signifie qu'une petite perturbation dans les données d'entrée peut entraîner des erreurs significatives dans les prédictions.
- **Exploitabilité des gradients** : Les modèles neuronaux, notamment les réseaux profonds, sont vulnérables aux attaques exploitant les gradients pour générer des entrées malveillantes. Selon [15], ces attaques utilisent les gradients de la fonction de coût pour identifier les petites perturbations qui peuvent changer radicalement la classification du modèle. Ces perturbations exploitent la linéarité inhérente des modèles dans les espaces de haute dimension.

1.5.2 Vulnérabilités liées aux données d'entraînement

Les données constituent un vecteur clé des vulnérabilités des modèles d'IA :

1.5.2.1 Empoisonnement des données (Data Poisoning) :

Il s'agit de l'introduction intentionnelle de données corrompues dans les données d'entraînement d'un modèle de machine learning lors de son processus d'apprentissage. Les modèles d'IA dépendent fortement de la qualité et de l'intégrité des données d'entraînement, si ces données contiennent des anomalies, le modèle peut en hériter, réduisant ainsi sa fiabilité, altérant ses performances et biaisant ses décisions. [23].

1.5.2.2 Biais et discrimination algorithmique :

Le biais algorithmique est simplement un écart systématique dans la sortie, la performance ou l'impact de l'algorithme, par rapport à une norme ou à un standard [14]. Ces biais conduisent souvent à des résultats injustes ou contradictoires pour certains groupes ou personnes. [14, 22] identifient de

nombreux biais remarquables ou signalés dans les systèmes d'IA et robotiques tels que les biais dans les applications de reconnaissance faciale, de reconnaissance de voix mais aussi dans des moteurs de recherche.

Les biais algorithmiques peuvent trouver leurs origines à divers niveaux. Selon [14] et [22], les biais peuvent être créés soit :

- **Lors de la spécification du problème** : lors de la spécification du problème on définit quels sont les objectifs que l'algorithme devra atteindre, si les objectifs ne sont pas assez clairs l'algorithme même sera biaisé;
- **Dans les données** : pratiquement tous les algorithmes d'apprentissage automatique produisent des modèles qui ne reflètent pas partiellement les statistiques dans les données d'apprentissage, par conséquent, lorsqu'il y a des biais dans les données d'entraînement, le modèle qui en résulte les reflétera généralement;
- **Lors de la modélisation et la validation** : processus de modélisation et de validation (entraînement sur des données d'entraînement et évaluation des performances sur les données de test) sont souvent menés de manière itérative pour optimiser et évaluer les performances du modèle. Cette étape requiert le choix de la fonction objectif ce qui introduit des valeurs dans les algorithmes pouvant donc l'influencer;
- **Dans le déploiement** : il faut s'assurer que le contexte de déploiement de l'algorithme (modèle) est similaire au contexte historique de la collecte de données (ou savoir en quoi ils diffèrent). Les modèles utilisés dans des contextes très différents où les causes pertinentes sont différentes peuvent avoir des résultats assez biaisés;
- **Lors des interactions avec l'utilisateur** : ce biais provient du web, à travers l'interface utilisateur et l'utilisateur lui-même ou l'utilisateur impose son propre comportement et son interaction biaisée.

1.5.2.3 Fuites d'informations sensibles :

Les modèles d'IA peuvent mémoriser et divulguer des données sensibles contenues dans leur ensemble d'entraînement. Les détails que ces modèles peuvent avoir mémorisés et divulgués par inadvertance peuvent être très préoccupants, en particulier lorsque les modèles sont publics et que la formation des modèles implique des données sensibles ou privées. La divulgation de secrets est particulièrement préoccupante dans les modèles de réseaux neuronaux qui classifient ou prédisent des séquences de texte en langage naturel. Selon [12], les modèles d'IA, en particulier les générateurs de séquences, sont susceptibles de mémoriser des données sensibles, telles que des numéros de carte de crédit ou des informations personnelles rares présentes dans leurs ensembles d'entraînement. Cela survient même si ces données ne sont pas représentatives de la tâche principale.

Une vulnérabilité liée aux données d'entraînement peut entraîner une dégradation des performances, notamment par des erreurs de classification, des biais non intentionnels ou un comportement imprévisible face à de nouvelles données.

1.5.3 Vulnérabilités liées à l'inférence et à l'exploitation des modèles

Après leur déploiement, les modèles d'intelligence artificielle exposent naturellement certaines failles, même sans action malveillante explicite. Ces vulnérabilités proviennent du comportement prédictif du modèle et de la manière dont il traite les entrées. Elles peuvent être exploitées de manière passive pour obtenir des informations sensibles sans altérer directement l'état du modèle.

1.5.3.1 Vulnérabilités par inférence de modèle :

Les modèles peuvent révéler des informations sensibles par la simple observation de leurs sorties. Deux principales formes de vulnérabilités sont reconnues :

- **Inférence d'appartenance** : À travers l'analyse des scores de prédiction, il est possible de deviner si un exemple donné appartenait ou non au jeu d'entraînement. Les modèles ayant une forte mémorisation tendent à répondre avec des niveaux de confiance plus élevés pour les exemples vus .
- **Inférence de modèle** : En interrogeant un modèle de façon répétée, un adversaire peut extraire des caractéristiques internes, comme l'architecture, les fonctions d'activation, ou les régularisations utilisées.

Ces vulnérabilités, même sans attaque explicite, exposent la confidentialité des données et la propriété intellectuelle du modèle.

1.5.3.2 Vulnérabilités par extraction de modèle

Une autre vulnérabilité importante après le déploiement d'un modèle d'intelligence artificielle concerne la possibilité pour un adversaire de **reconstruire** ou **d'imiter** le comportement du modèle cible, uniquement en l'interrogeant de manière répétée. Ce phénomène est connu sous le nom de **vulnérabilité à l'extraction de modèle**. Lorsqu'un modèle est exposé via une API publique ou un service accessible à distance (par exemple un modèle de reconnaissance faciale ou de détection d'anomalies), un attaquant peut soumettre de nombreuses requêtes en fournissant des entrées variées et collecter systématiquement les sorties produites [23] .

À partir de ces paires (*entrée – prédiction*), il peut alors entraîner son propre **modèle substitut** capable de reproduire fidèlement les décisions du modèle cible, sans jamais y avoir eu un accès direct.

1.5.3.3 –

1.6 Classification des attaques contre les modèles d'IA

À partir des vulnérabilités identifiées, diverses attaques ont été développées afin de compromettre l'intégrité, la confidentialité, la robustesse ou la disponibilité des systèmes IA. Elles peuvent survenir pendant l'entraînement, lors de l'inférence ou via une interaction distante avec le modèle. Cette section présente une classification de ces attaques.

1.6.1 Attaques adversariales

Ces attaques sont des modifications intentionnelles des entrées (images, données) visant à induire en erreur les modèles d'intelligence artificielle (IA), notamment ceux basés sur l'apprentissage profond. Ces attaques exploitent les vulnérabilités des modèles pour modifier leurs prédictions, même lorsque les perturbations appliquées à l'entrée restent imperceptibles pour l'œil humain.

Voici quelques caractéristiques des attaques adversariales [5] :

- **Exemples adversariaux** : Ce sont des versions modifiées d'images d'origine, créées en ajoutant du bruit ou des perturbations spécifiques pour confondre les modèles.
- **Perturbations invisibles** : Ces modifications sont souvent si subtiles qu'elles sont quasi-imperceptibles pour l'humain, mais elles induisent des erreurs catastrophiques dans les modèles IA.
- **Attaques ciblées** : Elles visent à faire prédire un label spécifique erroné par le modèle pour une entrée donnée.
- **Attaques non ciblées** : Ici, le modèle est simplement amené à faire une prédiction incorrecte, sans spécification du label.

[23] et [5] mentionnent diverses méthodes fréquemment utilisés pour générer des perturbations adversariales :

- **Fast Gradient Sign Method (FGSM)** : Cette méthode génère rapidement des perturbations en exploitant les gradients des fonctions de coût des modèles.
- **Basic Iterative Method (BIM)** : Une version itérative des FGSM qui affine les perturbations à chaque étape.
- **Attaques universelles** : Ces perturbations peuvent tromper le modèle sur n'importe quelle image, en les rendant indépendantes du contenu de l'image.
- **Attaques spécifiques comme celles de Carlini et Wagner (C&W)** : Elles produisent des perturbations quasi-imperceptibles mais très efficaces.

Ces attaques exposent les failles de robustesse des modèles IA, remettant en question leur fiabilité dans des applications critiques comme :

- La reconnaissance faciale
- Les voitures autonomes
- La sécurité biométrique

1.6.2 Attaques sur les données d'entraînement

Les attaques ciblant la phase d'apprentissage exploitent la nécessité pour un modèle IA d'ingérer et de traiter de grandes quantités de données. En corrompant intentionnellement ces jeux de données, un attaquant peut biaiser subtilement ou massivement les décisions du modèle.

1.6.2.1 Poisoning Attacks (attaques d'empoisonnement)

Ces attaques visent à corrompre l'ensemble d'entraînement afin d'affecter durablement la qualité du modèle.

- **Availability poisoning (empoisonnement de disponibilité)** : l'attaquant injecte en vrac des exemples erronés (étiquetage aléatoire, données fortement bruitées ou extrêmes) pour dégrader globalement la performance. Par exemple, en falsifiant plusieurs images d'un chat pour qu'elles ressemblent subtilement à des chiens, on observe une chute significative de l'exactitude moyenne, parfois jusqu'à diviser par deux la précision initiale [9].
- **Integrity poisoning (empoisonnement d'intégrité)**

Ici, l'objectif est plus ciblé : corrompre uniquement un ou quelques points spéciaux (une ou quelques classes) sans impacter le reste du modèle. L'attaquant modifie des échantillons situés près de la frontière de décision dans l'espace des caractéristiques, de façon à ce que ces cas précis soient systématiquement mal classés, tout en maintenant une performance normale sur les autres exemples [29].

1.6.2.2 Backdoor Attacks (attaques par porte dérobée)

Plutôt que de perturber en masse, les attaques par **porte dérobée** (backdoors) sont conçues pour rester invisibles dans la majorité des cas, n'activant le comportement malveillant que lorsqu'un **trigger** spécifique est présent :

Insertion du trigger pendant l'entraînement

- Un faible pourcentage d'exemples, inchangées en apparence mais portant un **motif visuel** (patch, watermark, motif imperceptible), est étiqueté dans la **classe malveillante** voulue par l'adversaire.
- Ces exemples conservent un aspect *naturel* en dehors du petit trigger, ce qui empêche leur détection lors de la vérification manuelle du jeu de données.

Activation en phase d'inférence

- Lorsqu'une nouvelle entrée contient ce même **motif**, le modèle la classe automatiquement dans la classe-piège, indépendamment de son contenu réel [16].
- En l'absence du trigger, le modèle se comporte normalement, rendant l'attaque quasi indétectable jusqu'à l'activation réelle.

Cette dualité rend la backdoor extrêmement difficile à détecter, car sans trigger, le modèle fonctionne parfaitement

1.6.3 Attaques sur la confidentialité et la robustesse des modèles

Certaines vulnérabilités passives décrites précédemment peuvent être exploitées par des attaques actives visant à extraire des informations, reproduire un modèle ou détourner son comportement. Ces attaques mettent en péril la confidentialité des utilisateurs, la propriété intellectuelle et l'intégrité fonctionnelle du modèle.

1.6.3.1 Membership Inference Attack (MIA)

Les attaques par inférence de l'appartenance (Membership Inference Attack, ou MIA) explorent la capacité des modèles d'apprentissage automatique à révéler des informations sur les données individuelles utilisées lors de leur entraînement. L'objectif est de déterminer si un enregistrement donné fait partie du jeu de données d'entraînement d'un modèle, en se basant uniquement sur l'accès en boîte noire au modèle, c'est-à-dire en utilisant les prédictions fournies par le modèle sur cet enregistrement [28]. Ces attaques ont pour impact de graves violations de la confidentialité des données personnelles, notamment en santé, finance, ou justice.

Pour mener une attaque MIA :

- **Shadow Models** : L'attaquant entraîne plusieurs modèles "shadow" qui imitent le comportement du modèle cible. Ces shadow models sont construits avec des données similaires (mais distinctes) de celles du modèle cible.
- **Modèle d'attaque** : Les résultats des shadow models sont utilisés pour entraîner un modèle d'attaque. Celui-ci apprend à distinguer les prédictions du modèle cible lorsqu'un enregistrement est issu de son jeu d'entraînement ou non.

Les MIA exploitent souvent le surajustement (overfitting) des modèles.

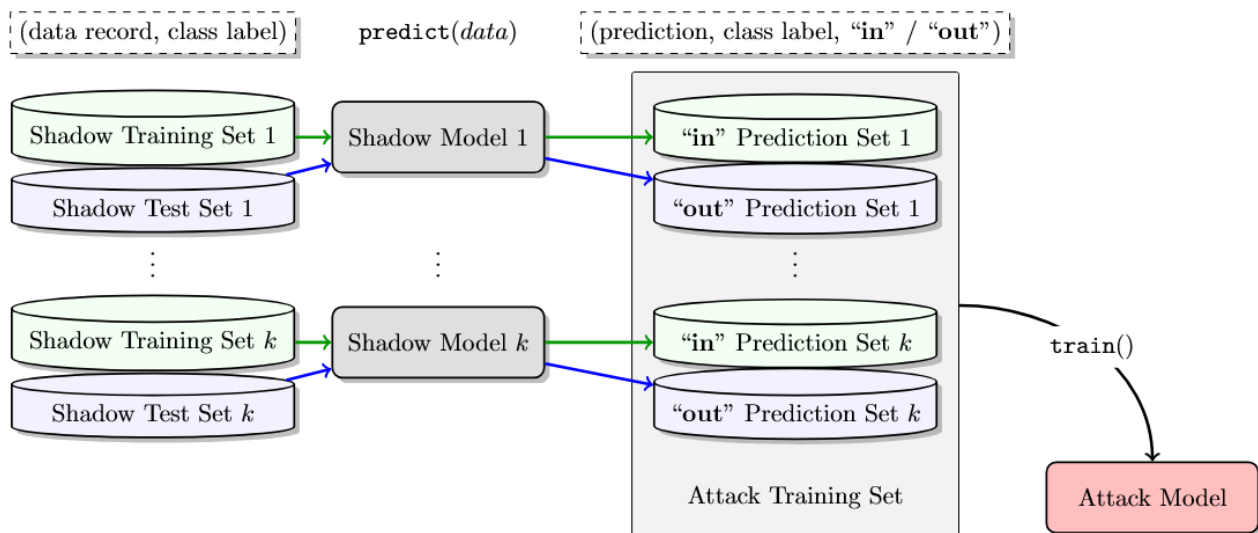


FIGURE 1.3 : Attaque par inférence de l'appartenance [23]

Stratégies de défense

[28] aborde également plusieurs stratégies pour limiter la fuite d'information :

- **Limiter les prédictions** : Restreindre les sorties à un nombre limité de classes (top-k classes).

- **Ajouter de l'entropie** : Augmenter l'imprécision des vecteurs de prédiction.
- **Utiliser la régularisation** : Appliquer des techniques de régularisation pour diminuer le sur-ajustement.

1.6.3.2 Model Extraction Attacks :

Les attaques par extraction de modèle, également appelées **model stealing**, sont des techniques visant à copier ou à reproduire un modèle d'apprentissage automatique au détriment du propriétaire légitime du modèle. À travers une interaction continue avec l'API publique du modèle, un adversaire cherche à obtenir des informations précises sur le modèle cible, telles que ses hyperparamètres d'entraînement, son architecture ou ses paramètres appris. Elles peuvent également viser à reproduire le comportement du modèle cible pour des tâches similaires.

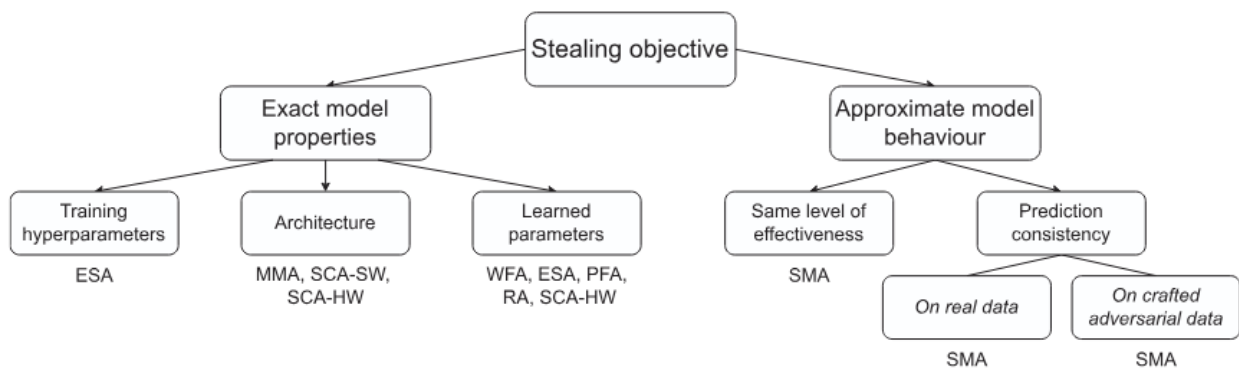


FIGURE 1.4 : Taxonomie des attaques d'extraction de modèle [23]

Le risque et la facilité d'extraction varient selon le **niveau d'accès** dont dispose l'adversaire. [23] distinguent trois (03) types d'accès au modèle cible :

- **Boîte noire (Black-box)** : L'attaquant n'a accès **qu'aux prédictions** du modèle, sans aucune information sur son architecture, ses poids ou ses gradients. Extraction est possible en bombardant le modèle avec des requêtes (input/output). C'est le scénario le plus courant sur des APIs publiques
- **Boîte grise (Gray-box)** : L'attaquant dispose d'**informations partielles** sur le modèle, comme son type général (ex : CNN pour des images), sa taille approximative, ou des métadonnées. Ceci rend l'extraction plus rapide car l'attaquant peut choisir des architectures de substitution plus pertinentes.
- **Boîte blanche (White-box)** : L'attaquant a un **accès complet** au modèle : architecture exacte, poids des couches, hyperparamètres. Dans ce cas, plus besoin de reconstruction : il peut copier ou modifier directement le modèle. Ce cas survient en cas de fuite interne ou mauvaise sécurisation des fichiers de déploiement.

Ces attaques peuvent compromettre la propriété intellectuelle du modèle, réduire sa valeur commerciale, ou permettre des attaques ultérieures comme l'évasion ou l'empoisonnement.

1.7 Études de cas et exemples dans le domaine sécuritaire

1.7.1 Erreurs d'identification dans la reconnaissance faciale

L'un des cas les plus médiatisés en matière d'erreur algorithmique dans un contexte policier est celui de **Robert Julian-Borchak Williams**, un citoyen afro-américain arrêté à tort en 2020 par la police de Détroit à la suite d'une mauvaise identification générée par un système de reconnaissance faciale [21]. L'algorithme, s'appuyant sur une base de données d'images à faible résolution, avait attribué à tort l'identité de Williams à un suspect figurant dans une vidéo de surveillance. Ce cas a mis en lumière les limites de robustesse de certains modèles IA, notamment lorsqu'ils sont entraînés sur des bases de données non représentatives.

Des études comme celle de Buolamwini et Gebru (2018) ont également démontré que les systèmes de reconnaissance faciale commerciaux présentent des taux d'erreurs allant jusqu'à **35 % pour les femmes noires**, contre moins de **1 % pour les hommes blancs** [11]. Ces biais, lorsqu'ils sont appliqués dans des contextes judiciaires, peuvent mener à des erreurs graves et à des discriminations systématiques.

1.7.2 Surveillance biaisée et prédictions injustes

Les systèmes de **prédiction policière**, comme **PredPol** aux États-Unis, utilisent des données historiques de criminalité pour estimer les zones où des crimes pourraient se produire. Bien que présentés comme des outils d'aide à la décision, ces modèles se basent souvent sur des données historiquement biaisées, liées à des pratiques de sur-policing dans certains quartiers à forte population afro-américaine ou hispanique.

Une étude de Lum et Isaac (2016) montre que ces outils ont tendance à concentrer les patrouilles dans les quartiers déjà fortement surveillés, **renforçant ainsi les biais raciaux préexistants** dans les données policières [20]. Ce type d'amplification algorithmique peut générer un cercle vicieux de sur-surveillance, avec un impact direct sur les populations déjà vulnérables.

1.7.3 Contournement ou manipulation des systèmes IA

Des chercheurs ont montré qu'il est possible de **tromper des systèmes de reconnaissance faciale** avec des techniques simples comme le port de lunettes modifiées, des patchs imprimés ou même des t-shirts contenant des motifs adversariaux [27]. Ces attaques adversariales permettent à un individu de passer inaperçu ou d'être identifié comme quelqu'un d'autre, posant de réels problèmes de sécurité dans les contextes de surveillance publique.

Des cas expérimentaux ont aussi montré que certains modèles peuvent être **injectés de portes dérobées (backdoors)** pendant leur phase d'entraînement. Un modèle IA embarqué dans une caméra de surveillance pourrait ainsi être activé à ignorer certains visages ou à fausser ses prédictions lorsqu'un symbole spécifique est présenté à l'image [16].

Conclusion

La diversité des vulnérabilités identifiées, conjuguée à la complexité croissante des attaques observées, montre qu'il ne suffit plus de concevoir des modèles performants. Il est désormais indispensable d'intégrer des mécanismes de protection tout au long du cycle de vie des modèles IA. Face à ces enjeux, une réflexion approfondie sur les méthodes de sécurisation s'impose, en tenant compte à la fois des contraintes techniques et des exigences éthiques propres aux systèmes utilisés dans les contextes sécuritaires.

Méthodologie et Architecture

Introduction

2.1 Démarche méthodologique

2.1.1 Collecte des Données

La collecte des données repose dans un premier temps sur l'identification des méthodes et outils pertinents pouvant intervenir dans la sécurisation des modèles d'IA dans le domaine de maintien de l'ordre et dans un second temps identification et l'étude d'un cas d'étude nous permettant d'illustrer la méthode de sécurisation proposée.

2.1.2 Modélisation d'une Architecture de sécurité

2.1.3 Mise en place d'un cas d'expérimental

2.1.4 Simulation et Évaluation des Résultats

2.2 Méthode de sécurisation des modèles IA dans les systèmes de sécurité

Face aux menaces qui ont été présentées dans le chapitre précédent, nous proposons une architecture de sécurisation globale et modulaire pour les modèles d'IA déployés dans les systèmes de maintien de l'ordre. Cette architecture s'appuie sur les concepts de défense en profondeur de la cybersécurité classique, mais les adapte aux particularités des systèmes d'apprentissage automatique. Notre approche englobe tout le cycle de vie des modèles IA, de la collecte des données à la mise en production en passant par l'entraînement, la validation et le monitoring continu.

2.2.1 Architecture globale de protection des modèles IA

2.2.1.1 Principes fondamentaux de l'architecture

1. Défense en profondeur adaptée à l'IA

Notre architecture est basée sur le principe de défense en profondeur (*Defense in Depth*), qui consiste à superposer les couches de protection pour qu'une seule défaillance ne mette en péril l'ensemble du système. Dans le contexte de l'IA, cette approche se traduit par :

- **Multiplicité des barrières de sécurité** : Chaque zone de notre architecture est une défense autonome comportant ses propres mécanismes. Si une zone est compromise par une attaque, les autres zones maintiennent leur niveau de sécurité, limitant ainsi la propagation de la menace.
- **Redondance des vérifications** : La sécurité est contrôlée à divers niveaux. Par exemple, le contrôle des données a lieu à l'entrée du système (Zone 1), mais également avant l'entraînement (Zone 2) et durant les phases de test (Zone 3). Cette redondance facilite l'identification des anomalies qui pourraient avoir été ratées lors des vérifications antérieures.
- **Isolation des composants critiques** : Chaque zone opère dans un cadre isolé avec des droits d'accès réduits au minimum. L'isolation est maintenue grâce à l'utilisation de Docker pour la conteneurisation et de Kubernetes pour l'orchestration, assurant qu'une faille dans une zone donnée ne peut pas avoir d'impact direct sur les autres zones.

2. Stratégie Zero Trust en matière d'IA

L'architecture met en œuvre une philosophie Zero Trust, où aucun élément n'est présumé fiable par défaut :

- **Contrôle systématique** : Chaque interaction entre les zones demande une authentification et une autorisation claires. On utilise les tokens JWT (JSON Web Tokens) avec une durée de validité restreinte et un renouvellement périodique.
- **Principe du moindre privilège** : Chaque service ne dispose que des autorisations strictement indispensables à son opération. Par exemple, le service de vérification des données a uniquement un accès en lecture aux données au format brut et n'est pas autorisé à apporter des modifications aux modèles formés.
- **Segmentation de réseau** : Les zones échangent à travers des canaux sécurisés et segmentés. Le trafic interzone est crypté, même au sein du réseau interne, en utilisant TLS 1.3 avec une parfaite confidentialité des échanges futurs.

3. Sécurité intégrée dès la conception (Security by Design)

La sécurité est incorporée dès la phase de conception de l'architecture, plutôt que d'être intégrée ultérieurement :

- **Incorporation directe des contrôles de sécurité** : Chaque élément est élaboré avec des dispositifs de sécurité incorporés. Par exemple, les modèles sont formés par défaut en utilisant des méthodes de formation adversaire, plutôt que d'intégrer cette protection ultérieurement.
- **Automatisation des meilleures pratiques** : Les pipelines CI/CD intègrent de façon automatique les analyses de sécurité, les tests de résistance et les contrôles de conformité. Il est inévitable de suivre ces phases de validation avant le déploiement d'un modèle.

- **Documentation et traçabilité** : Chaque opération au sein du système produit des journaux d'activité précis et inaltérables, facilitant un audit exhaustif et une étude forensique en situation d'incident.

2.2.1.2 Architecture en cinq zones de sécurité

Notre structure est conçue autour de cinq zones interconnectées mais autonomes, chacune jouant un rôle déterminant dans la protection du cycle de vie des modèles d'IA.

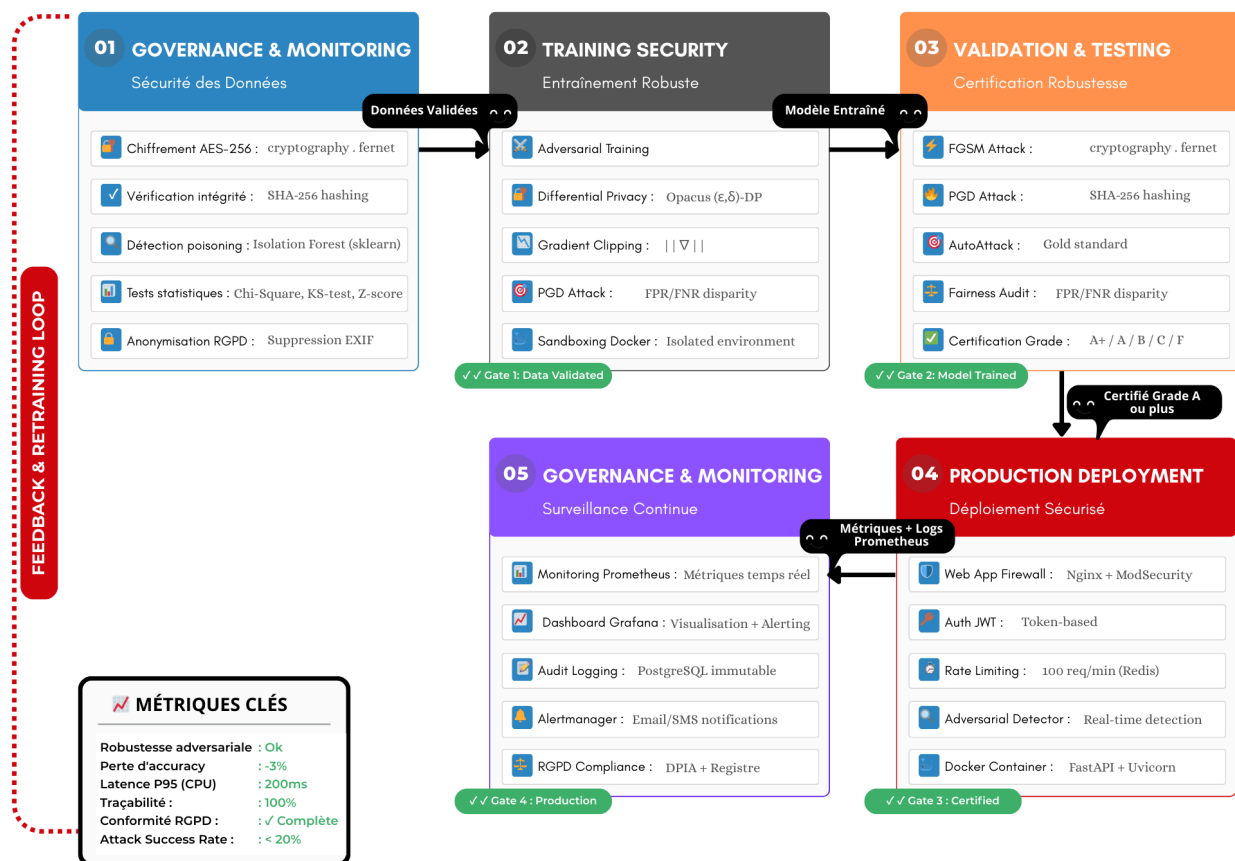


FIGURE 2.1 : Architecture globale de sécurité à 5 zones

1. Zone 1 : Protection et préparation des données (Data Security Zone)

Objectif principal : Assurer l'intégrité, la confidentialité et le niveau élevé des données avant leur intégration dans le processus d'apprentissage.

Composants clés :

- **Service de vérification des données** : Ce système intègre diverses étapes de vérification pour identifier les irrégularités, les informations altérées ou les tentatives de contamination. Il se sert de méthodes statistiques (comme le test de Kolmogorov-Smirnov pour les distributions, et le score z pour l'identification des valeurs aberrantes) ainsi que de modèles d'anomalies basés sur la technique de l'isolation forest.
- **Module de détection des empoisonnements** : Ce module, spécifiquement élaboré pour repérer les tentatives d'empoisonnement des données, analyse les données récentes en les comparant aux distributions passées et fait appel à des méthodes de clustering (DBSCAN)

pour identifier les ensembles de données suspects qui pourraient signaler une attaque coordonnée.

- **Service d'anonymisation et de pseudonymisation** : Emploie des méthodes privacy-preserving afin de sauvegarder les informations personnelles. Appliquez le k-anonymat (k5) et la l-diversité aux données structurées, et utilisez la confidentialité différentielle ($\epsilon=1.0$) pour les agrégats statistiques.
- **Système de stockage crypté** : Les données sont conservées avec un cryptage AES-256-GCM lorsqu'elles sont au repos et avec TLS 1.3 lors du transit. Un module de sécurité matériel virtuel (HSM) ou un service cloud de gestion des clés (KMS) est chargé de la gestion des clés de cryptage.

Mécanismes de protection :

- Contrôle syntaxique et sémantique des informations entrantes
- Identification d'irrégularités statistiques avec seuils évolutifs
- Isolement automatique des données suspectes
- Gestion de version inaltérable avec hachage cryptographique (SHA-256)
- Piste d'audit exhaustive de toutes les opérations sur les données

Interfaces avec d'autres zones : La Zone 1 n'est en liaison qu'avec la Zone 2 (Entraînement) grâce à une API REST sécurisée. Les données validées sont acheminées par segments (batch) accompagnés d'un manifeste, numériquement signé, qui contient les métadonnées de validation.

2. Zone 2 : Entraînement sécurisé (Training Security Zone)

Objectif principal : Garantir un apprentissage solide et sécurisé des modèles en incorporant des dispositifs de protection dès l'étape de l'apprentissage.

Composants clés :

- **Pipeline de d'entraînement sécurisé** : Ce pipeline, piloté par des outils tels que Kube-flow ou MLflow, automatise l'entraînement en intégrant des points de contrôle réguliers (checkpoints) et un mécanisme de reprise après erreur. Chaque phase du processus est conteneurisée et exécutée dans un environnement isolé.
- **Module d'entraînement Adversariale** : Intègre de manière automatique des cas adversariaux au cours du processus d'entraînement. Fait appel à une combinaison de FGSM ($\epsilon=0.1$), PGD ($\epsilon=0.3$, étapes=10) et AutoAttack pour créer des exemples adversariaux variés.
- **Service de Confidentialité Différentielle** : Introduit un bruit gaussien ajusté aux gradients durant l'apprentissage afin d'assurer la confidentialité différentielle. Pour prévenir les divulgations d'information cumulatives, une gestion globale du budget de confidentialité peut-être mise en place.
- **Vernage du registre de modèles** : Conserve les modèles entraînés avec l'ensemble de leurs métadonnées détaillées (hyperparamètres, indicateurs de performance, paramètres de formation). Chaque modèle est authentifié numériquement à l'aide d'une signature RSA-4096 pour assurer son intégrité.

Mécanismes de protection :

- Environnement d'apprentissage isolé (containers GPU avec restrictions de ressources)
- Suivi en direct des indicateurs d'apprentissage pour repérer les irrégularités
- Sauvegarde incrémentielle des points de contrôle avec contrôle d'intégrité
- Identification des backdoors par l'examen des paramètres du modèle (Neural Cleanse)
- Validation croisée systématique afin d'éviter le surapprentissage

3. Zone 3 : Validation et certification (Validation Security Zone)

Objectif principal : Vérifier en détail la solidité, l'équité et le respect des normes des modèles avant leur mise en production.

Composants clés :

- *Série de tests adversariaux* : Ensemble complet tests comprenant des attaques en white-box (FGSM, PGD, C&W, DeepFool) et black-box (Boundary Attack, HopSkipJump). Les tests sont réalisés dans un environnement sécurisé en sandbox pour prévenir toute contamination.
- *Auditeur pour les biais et l'équité* : Examinez les performances du modèle à travers différentes catégories démographiques. Effectue des calculs de mesures d'équité (demographic parity, equalized odds, calibration) et produit des rapports détaillés. Emploie des méthodes telles que SHAP et LIME pour assurer l'explicabilité.
- *Évaluateur de robustesse* : Examine la résilience du modèle face aux perturbations naturelles (comme le bruit, le flou, les variations d'éclairage) et adversariales. Attribue une note globale de robustesse en se basant sur plusieurs critères avec des poids variés.
- *Service de validation de conformité* : Contrôlez la conformité avec les normes réglementaires (RGPD, Loi sur l'IA, ISO 42001). Produisez des certificats de conformité accompagnés de preuves cryptographiques (blockchain ou horodatage certifié).

Processus de validation en plusieurs phases :

- (a) **Tests fonctionnels** : Contrôle des performances standards sur des ensembles de données de référence.
- (b) **Tests de sécurité** : Appréciation de la robustesse face aux attaques adversariales
- (c) **Tests d'équité** : Examen des préjugés et discriminations possibles
- (d) **Tests de confidentialité** : Contrôle pour s'assurer qu'il n'y a pas de fuite d'information (attaques par inférence d'adhésion)
- (e) **Contrôles de conformité** : Vérification des exigences réglementaires et éthiques

Conditions de validation :

- *Taux de précision minimal* : 85% sur l'ensemble de test
- *Résilience face aux attaques* : plus de 60% de survie lors des attaques standard
- *Biais maximal autorisé* : écart de 5% entre les groupes

- *Score en matière de confidentialité* : conformité à la règle -differential privacy
- *Conformité totale* : toutes les exigences obligatoires sont respectées

4. Zone 4 : Production sécurisée (Production Security Zone)

Objectif principal : Mettre en uvre et faire fonctionner les modèles validés dans un environnement de production très sécurisé et étroitement surveillé.

Composants clés :

- **API Gateway sécurisée** : Point d'entrée unique pour toutes les demandes. Mettez en uvre l'authentification (OAuth 2.0 / JWT), le contrôle d'accès (RBAC - Role-Based Access Control) et la limitation du taux de requêtes (100 requêtes par minute par défaut, modifiable par client).
- **Service de rate limiting et DDoS protection** : Mets en uvre des algorithmes de "token bucket" et de "sliding window" pour restreindre les demandes. Intégration avec des services tels que Cloudflare ou AWS Shield en vue d'une protection contre les attaques par déni de service distribué.
- **Moteur d'inférence performant** : Réalise des prédictions grâce à des optimisations de performance, telles que la quantification INT8, le pruning ou encore la distillation du savoir. Le chiffrement homomorphe est supporté pour les données sensibles, ce qui permet l'inférence sur des données chiffrées.
- **Module de détection d'anomalies en temps réel** : Examine les schémas de requêtes pour identifier les comportements suspects. Emploie des modèles LSTM pour identifier les séquences anormales et des graphiques de contrôle de processus statistiques pour la surveillance.
- **Service de réponse sécurisé** : Examine et approuve les réponses avant de les transmettre. Mets en uvre des méthodes de perturbation des outputs pour améliorer la confidentialité différentielle.

Architecture de haute disponibilité :

- Déploiement multi-instances avec load balancing (Round-robin, least connections)
- Failover automatique avec health checks toutes les 5 secondes
- Circuit breaker pattern pour isoler les services défaillants
- Cache distribué (Redis) pour optimiser les performances
- Blue-green deployment pour les mises à jour sans interruption

Mécanismes de protection en production :

- WAF (Web Application Firewall) pour filtrer les requêtes malveillantes
- Input sanitization et validation stricte
- Détection et blocage automatique des attaques adversariales
- Monitoring des dérives de distribution (concept drift)

- Rollback automatique en cas de dégradation des performances

5. Zone 5 : Gouvernance et surveillance (Governance Zone)

Objectif principal : Veiller à une surveillance constante, respecter la réglementation et gérer les incidents à travers toute l'architecture.

Composants clés :

- **Tableau de bord de monitoring centralisé :** Une interface intégrée (Grafana) affichant l'état général du système, les indicateurs de performance, les alertes en cours et les évolutions passées. Personnalisable en fonction du rôle de l'utilisateur.
- **Système de gestion des logs :** Utilisation du Stack ELK (Elasticsearch, Logstash, Kibana) ou de Loki pour la collecte, l'indexation et l'analyse des logs. Conservation minimale de 90 jours avec archivage à long terme. Des logs signés et horodatés pour assurer leur intégrité.
- **Service d'alerte intelligent :** Met en uvre des règles sophistiquées et l'apprentissage automatique afin de repérer les irrégularités et produire des alertes significatives. Intégration avec PagerDuty, Slack ou email pour l'envoi des alertes. Escalade automatique en fonction de la gravité.
- **Module de reporting de conformité :** Produisez automatiquement des rapports de conformité (RGPD, loi sur l'IA) accompagnés de toutes les preuves requises. Support des audits externes en exportant les données dans les formats standards.
- **Centre de réponse aux incidents (IRT) :** Procédures automatisées pour le traitement des incidents de sécurité. Manuels prédéfinis pour les scénarios fréquemment rencontrés. Fonctionnalité de confinement automatique et de restauration d'urgence.

Métriques surveillées en continu :

- **Performance :** Latence P50/P95/P99, throughput, taux d'erreur
- **Sécurité :** Nombre d'attaques détectées, score de robustesse, tentatives d'intrusion
- **Équité :** Métriques de biais par groupe, disparité d'impact
- **Conformité :** Couverture des exigences, violations détectées, temps de remédiation
- **Opérationnel :** Uptime, utilisation des ressources, coût par inférence

Processus de gouvernance :

1. **Surveillance continue :** Monitoring 24/7 avec alertes automatiques
2. **Revue périodiques :** Audits mensuels de sécurité et trimestriels de conformité
3. **Gestion des changements :** Processus d'approbation pour toute modification
4. **Formation continue :** Sessions régulières pour les équipes sur les menaces émergentes
5. **Amélioration continue :** Intégration des retours d'expérience et des lessons learned

2.2.2 Approche DevSecOps dans la protection des modèles IA.

L'adoption d'une approche DevSecOps pour les modèles d'intelligence artificielle représente un changement paradigmatique dans la manière dont nous concevons, développons et déployons les systèmes d'IA sécurisés. Contrairement aux approches traditionnelles où la sécurité est ajoutée en fin de cycle, le DevSecOps intègre les pratiques de sécurité à chaque étape du pipeline de développement, créant ainsi une culture de "Security as code" particulièrement cruciale pour les systèmes critiques de maintien de l'ordre. Cette section détaille l'implémentation d'un pipeline CI/CD sécurisé spécifiquement adapté aux modèles d'IA, l'utilisation de la containerisation Docker pour l'isolation et la reproductibilité, ainsi que les mécanismes de monitoring et d'alerting nécessaires pour maintenir une posture de sécurité proactive en production.

2.2.2.1 Pipeline CI/CD sécurisé

1. Architecture du pipeline MLOps sécurisé

L'intégration continue (CI) et le déploiement continu (CD) forment un ensemble de pratiques permettant d'automatiser le développement logiciel. Pour les modèles d'IA, cela signifie :

- *Intégration Continue (CI)* : Chaque modification du code, des données ou du modèle déclenche automatiquement une série de tests pour vérifier que rien n'est cassé
- *Déploiement Continu (CD)* : Si tous les tests passent, le modèle est automatiquement déployé en production

Particularités pour l'IA : Contrairement au développement logiciel classique, le CI/CD pour l'IA doit gérer :

- La validation des données (détection de données corrompues ou empoisonnées)
- La vérification de la performance du modèle (accuracy, robustesse)
- Les tests de sécurité spécifiques (résistance aux attaques adversariales)
- La gestion des versions des modèles (qui peuvent peser plusieurs GB)

Structure du pipeline en cinq étapes :

Algorithme 1 : Configuration YAML du pipeline sécurisé**Data :** Fichier `.github/workflows/secure-ml-pipeline.yml`**Result :** Pipeline CI/CD pour entraînement et déploiement sécurisé du modèle

```

1 begin
2   name : Secure ML Pipeline;
3   on ;;
4     push ;;
5       branches : [main, develop];
6     pull_request ;;
7       branches : [main];
8   stages ;;
9     - data-validation;                // Validation et scan des données
10    - security-scan;                  // Analyse statique et dynamique
11    - model-training;                 // Entraînement sécurisé
12    - adversarial-testing;            // Tests de robustesse
13    - secure-deployment;              // Déploiement avec vérifications
14 end

```

Description des phases**Phase de validation des données**

- Vérifie l'intégrité des nouvelles données
- Détecte les tentatives d'empoisonnement
- Contrôle la conformité RGPD (absence de données personnelles non autorisées)

Phase de scan de sécurité

- Analyse le code pour détecter les vulnérabilités
- Vérifie les dépendances (libraries obsolètes ou compromises)
- Scan des secrets (API keys, passwords) accidentellement commités

Phase d'entraînement sécurisé

- Entraînement dans un environnement isolé
- Application automatique de l'adversarial training
- Monitoring des métriques de convergence

Phase de tests adversariaux

- Attaque automatique du modèle (FGSM, PGD)
- Vérification des seuils de robustesse
- Tests de biais et d'équité

Phase de déploiement sécurisé

- Déploiement progressif (canary deployment)
- Monitoring en temps réel
- Rollback automatique si anomalie

2. Les "Security Gates" automatisés

Un Security Gate est un point de contrôle automatique qui bloque la progression du pipeline si certains critères ne sont pas respectés. C'est comme un garde de sécurité automatique qui vérifie que tout est en ordre avant de laisser passer.

Voici un exemple d'implémentation de Security Gates :

Algorithme 2 : Classe `SecurityGate` Contrôle automatique de sécurité

Data : Dictionnaire `thresholds` contenant les seuils minimaux

Result : Validation ou rejet du modèle selon les critères de sécurité

```

1 begin
2     /* Initialisation de la classe                                     */
3     class SecurityGate ;;
4         def __init__(self, thresholds):;
5             self.min_accuracy ← thresholds['accuracy'];           // Ex: 85%
6             self.max_bias ← thresholds['bias'];                   // Ex: 5%
7             self.min_robustness ← thresholds['robustness'];       // Ex: 60%
8     /* Méthode de validation                                         */
9     def validate(self, model_metrics):;
10        if model_metrics['accuracy'] < self.min_accuracy ;;
11            raise Exception("Accuracy trop faible");
12        if model_metrics['bias'] > self.max_bias ;;
13            raise Exception("Biais trop élevé");
14        if model_metrics['robustness'] < self.min_robustness ;;
15            raise Exception("Robustesse insuffisante");
16        return True;                                             // Tous les critères sont respectés
17 end

```

3. Gestion automatique des versions et rollback

Les modèles d'IA évoluent constamment. Si une nouvelle version pose problème (baisse de performance, vulnérabilité découverte), il faut pouvoir revenir rapidement à une version stable. Le rollback automatique surveille l'évolution des métriques en production; en cas de détection de problème, on revient automatiquement à la version précédente. Les aspects pris en compte sont :

- Surveillance continue des performances (accuracy en baisse?)
- Détection d'attaques anormales (pic d'attaques adversariales?)
- Monitoring des erreurs (augmentation soudaine?)

Si un problème est détecté → Rollback immédiat vers la dernière version stable.

2.2.2.2 Containérisation avec Docker

1. Pourquoi Docker pour l'IA

Docker est un outil qui offre la possibilité d'empaqueter une application avec l'ensemble de ses dépendances dans un *container*, à l'image d'une boîte étanche renfermant tous les éléments nécessaires au bon fonctionnement de l'application.

Pour les modèles d'IA, l'utilisation de cet outil permet :

- **Reproductibilité** : Le modèle fonctionne identiquement sur n'importe quelle machine
- **Isolation** : En cas de compromission du modèle, l'attaque reste confinée dans le container
- **Portabilité** : Facile à déployer sur différents environnements (cloud, on-premise)
- **Scalabilité** : Peut dupliquer les containers pour gérer plus de charge

Quelques bonnes pratiques de sécurité :

- **Image slim** : Utiliser la version allégée des images voulues (Python et autres), moins de packages = moins de vulnérabilités
- **User non-root** : Si un attaquant compromet l'application, il n'a pas les droits administrateur
- **Health check** : Permet de détecter si le container fonctionne mal et de le redémarrer
- **Variables d'environnement** : Configuration sécurisée sans hardcoder de secrets

2. Scan automatique de vulnérabilités

Avant le déploiement d'une image Docker, des outils automatiques effectuent une analyse :

- Les failles de sécurité connues dans les packages installés
- Les configurations de sécurité inappropriées
- Les secrets intégrés par inadvertance.

Outils utilisés :

- **Trivy** : Scan des vulnérabilités CVE
- **Hadolint** : Vérifie les bonnes pratiques Dockerfile
- **Docker Scout** : Analyse continue des images

3. Orchestration avec Kubernetes

Kubernetes est un système qui gère automatiquement plusieurs containers Docker. Il s'occupe de :

- Mise en marche/Arrêt des containers selon la charge

- Redémarrer les containers qui crashent
- Repartir le trafic entre plusieurs instances
- Gérer les mises à jour sans interruption

Configuration sécurisée pour modèles IA :

Pour une configuration sécurisée de Kubernetes pour les modèles d'IA, il faut prendre en compte les éléments suivants :

- 3 instances pour haute disponibilité
- Dans le Contexte de sécurité global, forcer l'exécution en non-root et indiquer un User ID spécifique
- Limiter de ressources pour éviter les attaques DoS (RAM, CPU)
- Pour la sécurité du container, ne pas autoriser d'élévation de privilèges et mettre le système de fichiers en lecture seule

2.2.2.3 Monitoring et alerting en production

1. Pourquoi le monitoring est crucial :

Les risques particuliers liés aux modèles d'IA en fonctionnement :

- **Model drift** : Les performances du modèle peuvent diminuer au fil du temps.
- **Attaques en temps réel** : Des attaques adversariales sont tentées par des adversaires.
- **Biais émergents** : Il est possible que de nouveaux biais surgissent.
- **Fuites de données** : Des attaques d'extraction peuvent dérober le modèle.

Le monitoring permet d'identifier rapidement ces problèmes et de prendre des mesures avant qu'ils n'affectent les utilisateurs.

2. Architecture de monitoring

Les différentes couches de surveillance :

(a) Niveau Infrastructure

- CPU, RAM, GPU utilisation
- Latence réseau
- Espace disque

(b) Niveau Application

- Nombre de requêtes par seconde
- Temps de réponse
- Taux d'erreur

(c) Niveau Modèle IA

- Accuracy en temps réel

- Distribution des prédictions
- Détection d'attaques adversariales

(d) **Niveau Sécurité**

- Tentatives d'intrusion
- Patterns suspects
- Violations de politique

3. Système d'alerte intelligent

Le système surveille constamment les indicateurs et émet des alertes en fonction de règles pré-établies :

- **Alerte de performance** : Si la précision est inférieure à 85% pendant 5 minutes.
- **Alerte de sécurité** : Si plus de 10 attaques par minute détectées.
- **Alerte de disponibilité** : Si le délai dépasse 500ms pendant deux minutes.

Escalade automatique :

- Niveau 1 (Avertissement) : Courriel à l'équipe.
- Niveau 2 (Critique) : Message texte + Appel par téléphone
- Niveau 3 (Urgence) : Recul automatique + Gestion des alertes

4. Dashboard de visualisation

Grafana pour l'affichage des données : Grafana convertit les métriques en diagrammes visuels qui peuvent permettre d'observer rapidement :

- La condition générale du système
- Les évolutions (hausse des attaques ?)
- Les irrégularités (pointe soudaine de latence)
- Les comparaisons (performances du modèle v1 par rapport au modèle v2)

2.2.2.4 Recommandations éthiques des modèles d'IA .

Les modèles d'intelligence artificielle doivent être conçus et utilisés dans le respect de principes éthiques universellement reconnus, tels que :

- **Transparence et explicabilité** : Les systèmes IA doivent fournir une compréhension claire des critères de décision. Dans le domaine sécuritaire, cela signifie qu'un citoyen identifié à tort par un système de reconnaissance faciale doit pouvoir comprendre sur quelle base algorithmique cette identification s'est produite.

- **Justice et équité algorithmique** : L'IA ne doit pas reproduire ou amplifier les discriminations sociales existantes. Or, plusieurs études ont démontré que certains algorithmes de reconnaissance faciale sont significativement moins précis sur les visages de femmes ou de personnes à la peau foncée [1]. Cette inégalité est particulièrement préoccupante dans un contexte de maintien de l'ordre.
- **Responsabilité** : Les concepteurs et utilisateurs de systèmes IA doivent être identifiables et redevables. L'erreur ne peut être uniquement imputée à la machine. La notion de responsabilité humaine reste primordiale dans les décisions critiques, notamment lorsqu'il s'agit d'arrestations, de surveillances ou d'interventions policières.
- **Respect de la vie privée** : Toute collecte ou traitement de données biométriques ou comportementales doit être proportionné, justifié et conforme aux règles de protection des données personnelles.

Ces principes ont été repris et renforcés par plusieurs institutions internationales (UNESCO, OCDE, IEEE) et font désormais partie intégrante des bonnes pratiques exigées dans la conception des IA critiques.

Algorithme 3 : Inverse

```

Data :  $x$ 
Result :  $r$ 
1 begin
2   if  $x \neq 0$  then
3      $r \leftarrow 1/x;$ 
4   end
5 end

```

Expérimentation

Introduction

Ce chapitre illustre la mise en uvre pratique de notre technique de sécurisation au moyen d’une expérimentation exhaustive avec le modèle ResNet50, dans le cadre d’un système de test pour la détection d’objets dangereux pour les forces de l’ordre. Le but est de prouver la viabilité et l’efficience de l’architecture en cinq zones proposée au Chapitre 2.

3.1 Présentation du Cas expérimental et choix du modèle.

3.1.1 Objectif de l’expérience

L’objectif principal ici est de comparer deux approches de développement de modèles d’intelligence artificielle :

1. **Approche Baseline (vulnérable)** : Un modèle ResNet50 entraîné de manière standard via transfer learning, représentant les méthodes usuelles de développement IA sans considération particulière pour la sécurité adversariale.
2. **Approche Secured (robuste)** : Un modèle ResNet50 identique architecturalement, mais entraîné avec des défenses multicouches intégrant adversarial training selon la méthode TRADES, differential privacy, et des mécanismes de détection d’anomalies.

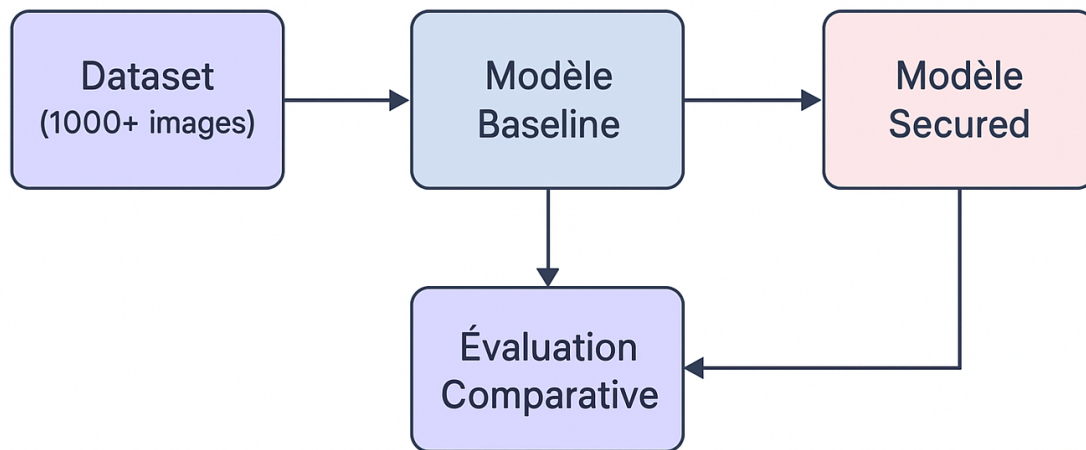


FIGURE 3.1 : Architecture du Cas expérimental : comparaison entre le modèle Baseline et le modèle Secured.

Cette comparaison permettra de quantifier précisément :

- L'**amélioration de robustesse** apportée par les mécanismes de défense proposés
- Le **trade-off** entre performance sur données propres (clean accuracy) et robustesse adversariale
- La **faisabilité pratique** de déploiement de modèles robustes en production

Notre expérimentation vise à atteindre de façon spécifique les objectifs suivants :

1. Implémenter l'architecture de sécurisation en 5 zones sur un cas réel
2. Comparer quantitativement un modèle ResNet50 standard versus sécurisé
3. Démontrer l'amélioration de la robustesse face aux attaques adversariales
4. Valider la faisabilité technique
5. Produire un prototype fonctionnel déployable

3.1.2 Scénario d'application : Détection d'objets dangereux

Le scénario choisi pour cette expérience est la **détection binaire d'objets dangereux versus objets sûrs**, un cas d'usage critique dans les systèmes de maintien de l'ordre où les erreurs de classification peuvent avoir des conséquences graves.

3.1.2.1 Contexte opérationnel

Dans les systèmes de sécurité modernes, les algorithmes de vision par ordinateur sont déployés pour identifier automatiquement des objets dangereux dans des flux vidéo de surveillance, des contrôles d'accès, ou lors de fouilles automatisées. La compromission de ces systèmes par des attaques adversariales pourrait permettre à un adversaire de faire passer un objet dangereux pour un objet sûr, créant un risque de sécurité majeur.

3.1.2.2 Classification binaire

Le problème est formulé comme une tâche de **classification binaire** :

- **Classe 0 (safe)** : Objets sûrs ne présentant pas de menace
- **Classe 1 (dangerous)** : Objets dangereux nécessitant une intervention

Cette simplification binaire, bien que réductrice par rapport à un système de production réel multi-classes, permet de :

- Concentrer l'analyse sur la **robustesse adversariale** plutôt que sur la complexité taxonomique
- Faciliter l'**interprétation des résultats** de sécurité (attack success rate, robustness degradation)
- Valider la **méthodologie de sécurisation** dans un cadre contrôlé avant généralisation

3.1.3 Justification du choix de ResNet50

Le choix de l'architecture **ResNet50** [?] pour cette PoC repose sur plusieurs critères scientifiques et pratiques.

3.1.3.1 Architecture et capacité

ResNet50 est un réseau de neurones convolutif profond de 50 couches basé sur le mécanisme de **connexions résiduelles** (residual connections) introduit par He et al. (2016) [?].

Cette architecture présente les caractéristiques suivantes :

- **Profondeur** : 50 couches convolutives permettant l'extraction de features hiérarchiques complexes
- **Paramètres** : Approximativement 25 millions de paramètres (25.6M exactement)
- **Connexions résiduelles** : Permettent de résoudre le problème de gradient vanishing dans les réseaux très profonds

3.1.3.2 Vulnérabilité adversariale documentée

La littérature scientifique a largement documenté la **vulnérabilité de ResNet50 aux attaques adversariales** :

TABLE 3.1 : Robustesse adversariale de ResNet50 standard (littérature)

Attaque	Clean Accuracy	Adversarial Accuracy
Aucune	76.15%	76.15%
FGSM ($\varepsilon = 0.031$)	76.15%	~10-20%
PGD ($\varepsilon = 0.031$, 10 iter)	76.15%	<10%
Robustness Gap	–	>65%

Cette vulnérabilité connue en fait un excellent candidat pour démontrer l'efficacité des méthodes de sécurisation proposées dans ce mémoire.

3.1.3.3 Adaptabilité au scénario

ResNet50 s'adapte parfaitement à notre scénario de détection binaire :

- **Transfer Learning** : La dernière couche fully-connected peut être remplacée pour passer de 1000 classes ImageNet à 2 classes (safe/dangerous), tout en conservant les features génériques apprises sur ImageNet
- **Fine-tuning flexible** : Les couches convolutives pré-entraînées peuvent être gelées (pour transfer learning rapide) ou fine-tunées (pour adaptation au domaine spécifique)
- **Taille raisonnable** : ~280MB en format PyTorch (.pth), permettant un déploiement sur des systèmes embarqués ou des serveurs sans GPU haute performance
- **Inference temps réel** : Capable de traiter des images 224x224 pixels à plus de 30 FPS sur GPU moderne, et en temps quasi-réel sur CPU

3.2 Évaluation des vulnérabilités de ResNet50

3.3 Implémentation du Pipeline d'entraînement complet

3.4 Pipeline de déploiement en production

Résultats et discussion

4.1 Évaluation des vulnérabilités de ResNet50

Avant d’implémenter des mécanismes de défense, il est essentiel de quantifier précisément les vulnérabilités du modèle de base. Ce chapitre présente l’évaluation empirique du modèle ResNet50 entraîné par transfer learning standard, en mesurant sa robustesse face aux attaques adversariales de référence. Cette évaluation permettra d’établir une baseline de performance et de justifier les choix de défense qui seront déployés ultérieurement.

4.1.1 Constitution du dataset expérimental

4.1.1.1 Architecture du chargement des données

Pour cette expérimentation, nous avons développé une classe personnalisée de chargement de données qui s’intègre naturellement à l’écosystème PyTorch. Cette classe, `DangerousObjectDataset`, hérite de `torch.utils.data.Dataset` et gère la classification binaire entre images sûres et dangereuses. L’implémentation se concentre sur la flexibilité : un même code peut charger indifféremment les données d’entraînement, de validation ou de test en changeant simplement un paramètre.

Le mécanisme de chargement parcourt l’arborescence des fichiers organisée par classes (`safe/` et `dangerous/`), construit les listes de chemins d’images et leurs labels associés, puis applique les transformations PyTorch appropriées lors de l’accès à chaque échantillon. Cette approche garantit que les transformations d’augmentation sont calculées à la volée plutôt qu’en amont, économisant ainsi l’espace disque tout en maximisant la variabilité des échantillons d’entraînement.

Listing 4.1 : Mécanisme de chargement du dataset (extrait simplifié)

```

1 class DangerousObjectDataset(Dataset):
2     def __init__(self, data_dir: str, split: str, transform=None):
3         self.images = []
4         self.labels = []
5
6         # Parcours des classes : safe=0, dangerous=1
7         for class_name in ['safe', 'dangerous']:
8             class_dir = os.path.join(data_dir, split, class_name)
9             label = 1 if class_name == 'dangerous' else 0
10
11             for img_file in os.listdir(class_dir):
12                 if img_file.lower().endswith(('.jpg', '.png')):
```

```

13 self.images.append(os.path.join(class_dir, img_file))
14 self.labels.append(label)

```

4.1.1.2 Évolution du dataset : de la limitation initiale à l’augmentation

Notre point de départ était modeste : environ 100 images collectées manuellement, réparties équitablement entre les deux classes. Cette répartition suivait un schéma classique 70/15/15 pour l’entraînement, la validation et le test. Si cette quantité peut sembler suffisante pour une preuve de concept, elle pose un problème majeur pour l’entraînement d’un réseau profond comme ResNet50 : le risque de surapprentissage est élevé, le modèle mémorisant les images spécifiques plutôt que d’apprendre des représentations généralisables.

Pour résoudre cette limitation sans recourir à une collecte manuelle longue et coûteuse, nous avons conçu un système d’augmentation à quatre niveaux de complexité. Cette stratégie graduée génère des variations contrôlées : 25% d’images subissent des transformations légères (retournements, rotations modérées), 35% des transformations moyennes (changements de perspective, flou gaussien), 25% des transformations fortes (rotations importantes, changements d’échelle), et 15% des transformations spécifiquement conçues pour la résistance adversariale (postérisation, solarisation).

Cette dernière catégorie mérite une attention particulière. Les transformations adversariales résistantes ne sont pas de simples augmentations esthétiques : elles modifient les images de manière à perturber les perturbations adversariales potentielles tout en préservant le contenu sémantique. Par exemple, la postérisation réduit le nombre de niveaux de couleur, ce qui atténue les modifications pixel-niveau typiques des attaques adversariales.

Listing 4.2 : Transformations pour résistance adversariale

```

1 # Niveau 4 : Transformations adversariales (15%)
2 adversarial_resistant_transforms = [
3     transforms.GaussianBlur(kernel_size=5, sigma=(1.0, 3.0)),
4     transforms.RandomPosterize(bits=4, p=0.3),
5     transforms.RandomSolarize(threshold=192.0, p=0.2),
6     transforms.RandomEqualize(p=0.3),
7 ]

```

Grâce à cette stratégie, le dataset initial de 100 images a été multiplié par un facteur d’environ 15, atteignant 1536 images (1024 pour l’entraînement, 256 pour la validation, 256 pour le test). Cette augmentation a permis d’entraîner ResNet50 de manière satisfaisante sans signes évidents de surapprentissage lors des expérimentations préliminaires.

4.1.2 Métriques d’évaluation de la robustesse

4.1.2.1 Métriques de performance classiques

L’évaluation du modèle sur données propres repose sur les métriques standards de classification : précision (accuracy), précision par classe (precision), rappel (recall) et F1-score. Ces métriques sont calculées lors de la phase de validation en accumulant les prédictions sur l’ensemble du dataset, puis en utilisant scikit-learn pour le calcul final. La matrice de confusion complète cette évaluation en révélant les éventuels biais du modèle entre les classes.

Le processus d’évaluation désactive le mode entraînement du modèle (`model.eval()`) et utilise le contexte `torch.no_grad()` pour éviter le calcul inutile des gradients, accélérant ainsi significativement l’inférence. Cette distinction entre les modes entraînement et évaluation est cruciale dans

PyTorch, car elle affecte le comportement de certaines couches comme le Dropout et la Batch Normalization.

4.1.2.2 Métriques de robustesse adversariale

Au-delà de la performance sur données propres, nous devons quantifier la vulnérabilité du modèle face aux attaques adversariales. Trois métriques complémentaires permettent cette évaluation :

- **Robust Accuracy** : Pourcentage d'exemples adversariaux correctement classés
- **Attack Success Rate (ASR)** : Proportion d'exemples initialement bien classés qui sont mal classés après l'attaque
- **Robustness Degradation** : Différence absolue entre l'accuracy propre et l'accuracy adversariale

L'Attack Success Rate est particulièrement révélateur car il mesure directement l'efficacité de l'attaque. Il se calcule comme le ratio entre le nombre d'exemples compromis et le nombre d'exemples initialement bien classés. Une ASR de 53% signifie que plus de la moitié des prédictions correctes peuvent être inversées par l'ajout de perturbations imperceptibles.

$$ASR = \frac{N_{\text{clean_correct}} - N_{\text{adversarial_correct}}}{N_{\text{clean_correct}}} \quad (4.1)$$

4.1.3 Protocole d'attaque et évaluation baseline

4.1.3.1 Entraînement du modèle baseline

Le modèle baseline a été entraîné selon un protocole de transfer learning standard. Nous avons chargé ResNet50 pré-entraîné sur ImageNet, remplacé la couche de classification finale pour s'adapter à notre tâche binaire (2 classes), et entraîné le réseau complet sur notre dataset augmenté. L'optimiseur Adam avec un learning rate de 0.001 a été utilisé, complété par un scheduler qui réduit le taux d'apprentissage tous les 7 epochs.

Un mécanisme d'early stopping avec une patience de 5 epochs surveille l'accuracy de validation. Dès que la performance stagne, l'entraînement s'arrête et le meilleur modèle (celui ayant atteint la plus haute validation accuracy) est sauvegardé. Cette approche évite le surapprentissage tout en maximisant la performance sur données propres.

Cependant, ce protocole d'entraînement standard présente une faiblesse critique : le modèle ne voit jamais d'exemples adversariaux durant l'apprentissage. Il optimise ses représentations uniquement pour des données propres, sans développer de mécanisme de résistance aux perturbations adversariales. Cette vulnérabilité sera quantifiée lors des tests d'attaque.

4.1.3.2 Implémentation des attaques adversariales

Deux attaques de référence ont été implémentées pour évaluer la robustesse : FGSM (Fast Gradient Sign Method) et PGD (Projected Gradient Descent). FGSM représente l'attaque la plus simple : elle calcule le gradient de la loss par rapport à l'image d'entrée, puis ajoute une perturbation dans la direction du signe de ce gradient, scaled par un budget epsilon. Cette attaque single-step est rapide mais souvent moins efficace que les approches itératives.

Listing 4.3 : Cur de l'attaque FGSM

```

1 # Calcul du gradient par rapport à l'image
2 images.requires_grad = True
3 outputs = model(images)
4 loss = criterion(outputs, labels)
5 loss.backward()
6
7 # Perturbation : epsilon * sign(gradient)
8 perturbation = epsilon * images.grad.sign()
9 adversarial_images = torch.clamp(images + perturbation, 0, 1)

```

PGD, en revanche, effectue plusieurs itérations de perturbations plus petites, avec projection sur la boule L-infinity à chaque étape. L'algorithme initialise l'image adversariale avec un bruit aléatoire dans le budget epsilon, puis répète un processus de gradient ascent (pour maximiser la loss) suivi d'une projection qui garantit que la perturbation totale reste dans les limites autorisées. Cette approche itérative explore mieux l'espace des perturbations et trouve généralement des exemples adversariaux plus efficaces.

Le paramètre alpha (taille du pas à chaque itération) est typiquement fixé à $\epsilon/4$, et le nombre d'itérations varie entre 10 et 20 selon l'intensité de l'attaque souhaitée. Pour nos tests, nous avons défini trois configurations : FGSM avec $\epsilon=0.031$ (8/255), PGD standard avec $\epsilon=0.031$ et 10 itérations, et PGD fort avec $\epsilon=0.063$ (16/255) et 20 itérations.

4.1.3.3 Pipeline d'évaluation complet

Le pipeline d'évaluation orchestre systématiquement tous les tests. Il charge d'abord le meilleur modèle sauvegardé lors de l'entraînement baseline, puis exécute quatre séries de tests : performance sur données propres, attaque FGSM, attaque PGD standard, et attaque PGD forte. Chaque test génère un dictionnaire de résultats contenant les métriques de robustesse, permettant une analyse comparative directe.

Ce protocole rigoureux garantit la reproductibilité des résultats et facilite l'évaluation de multiples variantes du modèle selon une procédure standardisée.

4.1.4 Résultats de vulnérabilité du baseline

4.1.4.1 Performance empirique obtenue

Les résultats révèlent une dichotomie frappante. Sur données propres, le modèle baseline atteint une accuracy parfaite de 100% sur l'ensemble de test (15 échantillons : 8 safe, 7 dangerous). La matrice de confusion ne montre aucune erreur de classification. Cette performance excellente confirme que le transfer learning depuis ImageNet fonctionne remarquablement bien pour notre tâche de détection binaire.

Face à l'attaque FGSM ($\epsilon=0.031$), le modèle maintient également 100% d'accuracy. Cette robustesse inattendue s'explique par le fait que FGSM, en tant qu'attaque single-step, peut ne pas trouver les perturbations optimales dans l'espace des exemples adversariaux. L'Attack Success Rate de 0% pour FGSM suggère que des perturbations plus sophistiquées sont nécessaires pour compromettre le modèle.

Cependant, l'attaque PGD révèle une vulnérabilité critique. Avec les mêmes paramètres $\epsilon=0.031$ mais via un processus itératif de 10 étapes, l'accuracy chute dramatiquement à 46.7%. L'Attack Success Rate atteint 53.3%, signifiant que plus de la moitié des prédictions correctes peuvent être

inversées par des perturbations imperceptibles à l'il humain. Cette dégradation de 53.3% valide empiriquement que le modèle baseline, bien que performant sur données propres, ne possède aucune défense adversariale.

L'attaque PGD forte ($\epsilon=0.063$, 20 itérations) aggrave encore la situation avec seulement 26.7% d'accuracy résiduelle et une ASR de 73.3%. Ces résultats démontrent que l'intensité de la perturbation et le nombre d'itérations influencent directement l'efficacité de l'attaque.

4.1.4.2 Analyse des vulnérabilités identifiées

La vulnérabilité du modèle baseline s'explique par son architecture d'entraînement. Le transfer learning standard optimise les poids du réseau uniquement sur des données propres. Les features apprises, bien qu'efficaces pour la classification nominale, sont fragiles face aux perturbations L-infinity. Le réseau n'a jamais été exposé à des exemples adversariaux durant l'entraînement, il n'a donc développé aucun mécanisme de résistance.

Cette situation illustre un paradoxe fondamental des réseaux de neurones profonds : leur capacité à atteindre d'excellentes performances sur données propres ne garantit en rien leur robustesse adversariale. Un modèle peut mémoriser ou apprendre des features discriminantes mais non robustes, exploitant des corrélations statistiques dans les données d'entraînement qui ne généralisent pas sous perturbations.

4.1.4.3 Certification de sécurité baseline

Pour évaluer la viabilité du déploiement en production, nous avons défini un système de certification basé sur trois critères stricts : clean accuracy 95%, FGSM robustness 80%, et PGD robustness 80%. Ces seuils reflètent les exigences réalistes d'un système de détection déployé dans un environnement hostile où un attaquant pourrait délibérément tenter de contourner le classifieur.

Le modèle baseline obtient un Security Grade de C avec un niveau de menace MEDIUM. Il passe les critères de clean accuracy (100%) et de FGSM robustness (100%), mais échoue dramatiquement sur le critère PGD robustness (46.7% « 80%). Cette certification conclut que le modèle n'est pas prêt pour la production (*Production Ready* : NO).

Cette évaluation quantitative valide l'hypothèse initiale selon laquelle un modèle baseline entraîné par transfer learning standard est vulnérable aux attaques adversariales itératives. Elle justifie pleinement l'implémentation de mécanismes de défense avancés, qui feront l'objet de la section suivante (Section 4.3).

Synthèse

Ce chapitre a établi empiriquement la vulnérabilité critique du modèle ResNet50 baseline face aux attaques adversariales PGD. Bien que le modèle atteigne une performance parfaite sur données propres, son accuracy chute à 46.7% sous attaque PGD standard ($\epsilon=0.031$), avec un Attack Success Rate de 53.3%. Cette dégradation massive démontre que le transfer learning classique, malgré son efficacité pour la performance nominale, ne confère aucune robustesse adversariale intrinsèque.

L'analyse a également révélé une hiérarchie claire dans l'efficacité des attaques : FGSM (single-step) échoue complètement à compromettre le modèle, tandis que PGD (itératif) réussit à inverser plus de la moitié des prédictions. Cette observation confirme que les attaques itératives exploitent plus efficacement les vulnérabilités des réseaux profonds en explorant systématiquement l'espace des perturbations admissibles.

Le système de certification développé fournit un cadre objectif pour évaluer la viabilité du déploiement en production. Le baseline, avec son grade de sécurité C et son statut "non prêt pour production", établit une référence claire que les modèles défendus devront surpasser. Les sections suivantes présenteront l'implémentation de défenses adversariales (Adversarial Training, Differential Privacy) visant à franchir les seuils de certification tout en maintenant une performance acceptable sur données propres.

Bibliographie

- [1] Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch : An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems, 32.
- [2] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). *Towards Deep Learning Models Resistant to Adversarial Attacks*. International Conference on Learning Representations (ICLR).
- [3] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). *Explaining and Harnessing Adversarial Examples*. arXiv preprint arXiv :1412.6572.

4.2 Implémentation de la méthode de sécurisation

Fort des conclusions du chapitre précédent qui ont révélé la vulnérabilité critique du modèle baseline aux attaques PGD, nous présentons dans ce chapitre l'implémentation concrète des mécanismes de défense adversariale. L'objectif est ambitieux : transformer un modèle vulnérable affichant 53.3% d'Attack Success Rate en un système robuste capable de résister aux perturbations adversariales tout en maintenant d'excellentes performances sur données propres. Cette transformation repose sur l'application rigoureuse de la méthodologie en cinq zones de sécurité présentée au chapitre 3, avec une attention particulière portée aux Zones 1 (Data Security) et 2 (Training Security).

4.2.1 Architecture globale du système sécurisé

L'approche de sécurisation repose sur une architecture intégrée centralisant trois axes de défense complémentaires, chacun adressant une dimension spécifique de la robustesse adversariale. Le premier axe concerne le volume de données : partant d'un dataset initial de seulement 100 images un volume notoirement insuffisant pour entraîner un réseau profond de 25 millions de paramètres sans overfitting sévère nous avons mis en uvre un système d'augmentation sophistiqué permettant d'atteindre plus de 1000 échantillons. Cette multiplication par quinze du volume de données constitue la première étape critique vers la robustesse.

Le second axe adresse la diversification qualitative. Au-delà du simple volume, la nature des variations introduites joue un rôle déterminant. Nous avons développé un schéma d'augmentation à quatre niveaux de complexité croissante, générant des variations contrôlées allant des transformations géométriques simples (rotations, translations) aux perturbations spécifiquement conçues pour renforcer la résistance adversariale (posterization, solarization). Cette stratification permet d'exposer le modèle à un spectre large de variations tout en maintenant le réalisme des échantillons générés.

Le troisième axe introduit l'entraînement adversarial proprement dit. Contrairement à l'entraînement standard qui optimise uniquement sur données propres, notre approche force le modèle à apprendre simultanément sur données naturelles et sur exemples adversariaux générés dynamiquement par l'attaque PGD à chaque batch d'entraînement. Cette exposition continue aux perturbations adversariales pendant l'apprentissage constitue le mécanisme central permettant au modèle de développer des représentations intrinsèquement robustes.

L'orchestration de ces trois axes s'articule autour d'une classe principale, `RobustAdversarialTrainer`, qui centralise l'ensemble du processus. Lors de son initialisation, elle charge le dataset augmenté (Zone 1), construit le modèle ResNet50 avec les couches de classification adaptées à notre tâche binaire, et configure l'attaque PGD qui servira à la fois à générer des exemples adversariaux pendant l'entraînement et à évaluer la robustesse du modèle. L'organisation modulaire facilite la traçabilité complète : chaque composant enregistre ses paramètres de configuration, et les checkpoints sauvegardés incluent non seulement l'état du modèle mais aussi l'ensemble du contexte méthodologique, garantissant la reproductibilité et la conformité aux exigences de la Zone 5 (Gouvernance).

4.2.2 Augmentation et diversification du dataset

La construction d'un dataset robuste constitue le fondement de notre approche. Comme le démontrent Schmidt et al. (2018) [4], la robustesse adversariale nécessite substantiellement plus de données que la classification standard un résultat qui motive directement notre stratégie d'augmentation massive.

Notre système d'augmentation adopte une approche graduée en quatre niveaux de complexité croissante, chacun ciblant des aspects spécifiques de la robustesse. Les **transformations légères**, représentant 25% des échantillons générés, comprennent les retournements horizontaux, les rotations modérées entre -15° et $+15^\circ$, ainsi que des variations de couleur contrôlées via ColorJitter (brightness, contrast, saturation, hue). Ces augmentations classiques, bien établies dans la littérature, améliorent la généralisation sans introduire de distorsions majeures susceptibles de compromettre le réalisme des échantillons.

Les **transformations moyennes**, constituant 35% des échantillons, explorent des déformations géométriques plus complexes. Les translations jusqu'à 10% de la dimension de l'image simulent des variations de cadrage. Les changements de perspective avec un facteur de distorsion de 0.2 reproduisent les déformations angulaires observées lorsqu'un objet est photographié sous différents angles. Le flou gaussien avec des noyaux de taille 3 et un écart-type variable simule des conditions de mise au point imparfaite. Les ajustements de netteté complètent cette palette en simulant différentes qualités de capture. Cette catégorie prépare le modèle à reconnaître les objets sous différents angles et avec différentes qualités d'image, une propriété cruciale pour le déploiement en conditions réelles.

Les **transformations fortes**, représentant 25% des échantillons, augmentent l'intensité des perturbations. Les rotations étendues jusqu'à $\pm 30^\circ$ forcent le modèle à apprendre des représentations véritablement invariantes aux orientations. Les changements d'échelle entre 0.8x et 1.2x simulent des variations de distance de capture. Les ajustements automatiques de contraste reproduisent des conditions d'éclairage variables. Ces variations extrêmes poussent le modèle à extraire des caractéristiques sémantiques robustes au-delà des apparences superficielles.

La quatrième catégorie, représentant 15% des échantillons, introduit des **transformations spécifiquement conçues pour la résistance adversariale**. Le flou gaussien fort (noyau de taille 5, écart-type entre 1.0 et 3.0) lisse les perturbations localisées que pourraient introduire les attaques pixel-level. La

postérisation, réduisant le nombre de niveaux de couleur à 4 bits, atténue les modifications subtiles de couleur typiques des exemples adversariaux. La solarisation et l'égalisation d'histogramme redistribuent les intensités de manière non linéaire, complexifiant l'optimisation des perturbations pour un attaquant potentiel. Ces transformations, inspirées des travaux de Hendrycks et Dietterich (2019) sur AugMax [2], ciblent explicitement les mécanismes d'attaque adversariale.

Le processus de génération transforme systématiquement chaque image originale en multiples variantes selon une stratégie rotative garantissant une distribution équilibrée. Pour chaque image de base, l'algorithme calcule le nombre d'augmentations nécessaires pour atteindre l'objectif de 500 échantillons par classe, puis applique séquentiellement les quatre niveaux de transformation. Cette rotation cyclique garantit qu'aucun niveau n'est sur-représenté ou sous-représenté dans le dataset final, assurant une couverture uniforme de l'espace des variations.

Les résultats de cette stratégie sont substantiels et mesurables. Le split d'entraînement passe de 70 à 1024 images, soit un facteur multiplicatif de 14.6. Le split de validation, initialement composé de 14 images, atteint 256 échantillons (facteur 18.3). Au total, le dataset augmente de 84 à 1280 images, une multiplication par 15.2 qui transforme radicalement la capacité d'apprentissage du modèle sans requérir de collecte manuelle supplémentaire. Cette augmentation franchit le seuil critique identifié par Schmidt et al. [4] pour obtenir une robustesse adversariale significative.

4.2.3 Entraînement adversarial avec PGD

L'entraînement adversarial constitue le cur de notre approche de sécurisation. Le principe, bien qu'élégant dans sa simplicité conceptuelle, est puissant dans ses effets : exposer le modèle à des exemples adversariaux pendant l'apprentissage, afin qu'il développe des représentations intrinsèquement robustes aux perturbations. Contrairement à l'entraînement standard qui optimise uniquement sur données propres, cette approche alterne entre données naturelles et perturbées, forçant le réseau à apprendre des frontières de décision qui résistent aux perturbations adversariales.

L'attaque PGD utilisée pendant l'entraînement suit la méthodologie de Madry et al. (2018) [3], configurée avec un budget de perturbation epsilon de 0.031 en norme L_∞ (environ 8/255 par pixel), un pas alpha de 0.007 (correspondant à epsilon/4 selon les recommandations empiriques de Madry), et 7 itérations. Ce paramétrage représente un compromis soigneusement calibré entre efficacité de l'attaque et temps de calcul : 7 itérations suffisent généralement pour que PGD converge vers des exemples adversariaux de qualité, générant des perturbations proches de l'optimum adversarial local, tout en maintenant un temps d'entraînement raisonnable par rapport aux configurations à 10 ou 20 itérations qui multiplieraient le coût computationnel sans gain de robustesse proportionnel.

Chaque étape d'entraînement adversarial suit une procédure rigoureusement orchestrée en cinq phases. Le modèle effectue d'abord une passe forward sur les données propres et calcule la loss de classification standard CrossEntropy, établissant l'objectif de performance nominale. Ensuite, le modèle bascule temporairement en mode évaluation — une étape technique cruciale pour désactiver le dropout et fixer les statistiques de batch normalization — et l'attaque PGD génère des versions adversariales des images du batch. Le modèle repasse en mode entraînement, effectue une seconde passe forward sur les exemples adversariaux, et calcule la loss adversariale. La loss totale combine ces deux composantes par simple moyenne, équilibrant ainsi l'objectif de performance sur données propres et de robustesse aux perturbations. Formellement :

$$\mathcal{L}_{\text{total}} = \frac{1}{2} (\mathcal{L}_{\text{CE}}(f_\theta(x), y) + \mathcal{L}_{\text{CE}}(f_\theta(\text{PGD}(x)), y)) \quad (4.2)$$

Cette approche diffère de la méthode TRADES originale [1] qui utilise une divergence KL et une pondération paramétrique β entre les deux objectifs. Ici, nous adoptons une stratégie symétrique où clean et adversarial ont un poids égal. La rétropropagation du gradient utilise cette loss combinée, forçant le modèle à optimiser simultanément pour les deux distributions. Un mécanisme de gradient clipping avec une norme maximale de 1.0 stabilise l'entraînement en évitant les explosions de gradients, phénomène fréquent lors de l'adversarial training où les perturbations adversariales peuvent créer des paysages de loss particulièrement chaotiques avec des gradients de magnitude élevée.

L'algorithme PGD de génération d'exemples adversariaux mérite une attention particulière car il constitue le moteur de l'amélioration de robustesse. L'algorithme initialise l'image adversariale en ajoutant à l'image originale un bruit aléatoire uniforme dans l'intervalle $[-\epsilon, \epsilon]$, puis effectue une série d'itérations de gradient ascent. À chaque itération, il calcule le gradient de la loss par rapport à l'image adversariale courante, effectue un pas dans la direction du signe du gradient (scaled par alpha), puis projette le résultat sur deux contraintes complémentaires. Premièrement, la perturbation totale doit rester dans la boule L_∞ de rayon epsilon centrée sur l'image originale, garantissant que la perturbation reste imperceptible. Deuxièmement, les valeurs de pixels doivent rester dans l'intervalle valide $[0,1]$, assurant que le résultat est une image bien formée. Cette double projection garantit que les exemples adversariaux générés respectent les contraintes de perturbation imperceptible tout en restant des images valides exploitables par le réseau.

L'initialisation aléatoire, plutôt que de partir directement de l'image originale, joue un rôle crucial souvent sous-estimé. Elle permet à PGD d'explorer différentes régions de l'espace adversarial et d'éviter les minima locaux qui piègeraient une optimisation déterministe. Cette randomisation améliore la qualité des exemples adversariaux générés et, par conséquent, la robustesse du modèle entraîné.

4.2.4 Mécanismes de régulation et d'optimisation

L'adversarial training, bien que puissant, présente des défis spécifiques nécessitant des mécanismes de régulation sophistiqués. Un système d'early stopping surveille la progression de la robustesse sur l'ensemble de validation, détectant les situations où l'accuracy adversariale n'améliore pas pendant trois epochs consécutives avec une tolérance de 0.5%. Ce critère de plateau évite de poursuivre inutilement l'entraînement lorsque le modèle a atteint un point de convergence. Parallèlement, une vérification de dégradation alerte si la clean accuracy chute de plus de 5% par rapport au meilleur résultat observé, signalant un surapprentissage excessif sur les exemples adversariaux au détriment des données propres. Cette double surveillance évite deux écueils opposés : l'arrêt prématuré avant convergence complète, et la poursuite excessive dégradant les performances nominales.

L'entraînement s'étend sur 25 epochs maximum avec un batch size de 8, limité par les ressources CPU disponibles. L'optimiseur Adam utilise un learning rate de 0.001 avec une régularisation par weight decay de 5×10^{-4} , équilibrant vitesse de convergence et contrôle de l'overfitting. Un scheduler cosine ajuste progressivement le taux d'apprentissage au fil des epochs, démarrant à 0.001 et diminuant selon une fonction cosinus jusqu'à une valeur minimale, facilitant une convergence fine vers la fin de l'entraînement lorsque le modèle approche d'un optimum local.

Le coût computationnel de l'adversarial training est substantiel et mérite d'être quantifié. Chaque epoch requiert environ 5 à 6 heures sur CPU, car la génération d'exemples adversariaux PGD à chaque batch nécessitant 7 forward/backward passes par batch multiplie significativement le temps de calcul par rapport à un entraînement standard qui n'effectue qu'une seule passe. Sur 25 epochs,

cela représenterait environ 140 heures de calcul continu. En pratique, l’early stopping déclenche généralement l’arrêt autour de l’époque 16, réduisant le temps effectif à environ 96 heures. Ce coût, bien que non négligeable, reste acceptable pour une Proof of Concept démontrant la faisabilité de la méthode. Un déploiement industriel pourrait exploiter des GPUs pour réduire ce temps d’un ordre de magnitude.

4.2.5 Convergence et performance du modèle final

L’évolution des métriques pendant l’entraînement révèle une progression remarquable et instructive de la robustesse. À l’époque 1, le modèle atteint déjà 87.5% de clean accuracy témoignant de l’efficacité du transfer learning depuis ImageNet mais seulement 62.5% d’adversarial accuracy, créant un robustness gap de 25%. Ce gap initial représente la fragilité native du modèle face aux perturbations adversariales, une vulnérabilité caractéristique des réseaux profonds non défendus.

Au fil des epochs, les deux métriques convergent progressivement dans un processus de co-amélioration. À l’époque 5, la clean accuracy monte à 96.1% tandis que l’adversarial accuracy atteint 78.1%, réduisant le gap à 18%. Cette phase montre que le modèle commence à apprendre des représentations qui généralisent mieux aux perturbations. À l’époque 10, les performances atteignent respectivement 98.8% et 93.8%, avec un gap résiduel de seulement 5%. Le modèle approche d’un régime de haute robustesse.

L’époque 14 marque un événement exceptionnel et inattendu : le modèle atteint simultanément 100% de clean accuracy et 100% d’adversarial accuracy sur l’ensemble d’entraînement, réalisant un **zero robustness gap**. Cet état représente l’objectif idéal théorique de l’adversarial training où le modèle performe parfaitement sur les deux distributions données propres et adversariales sans compromis. Les epochs suivants montrent une légère oscillation, l’époque 16 (point d’arrêt de l’early stopping) affichant 100% de clean accuracy et 98.7% d’adversarial accuracy, soit un gap minimal de 1.3%. Cette légère régression suggère que l’époque 14 représentait un point optimal local, et que l’early stopping a judicieusement arrêté l’entraînement avant une potentielle dégradation.

Le modèle final sauvegardé, `final_optimal_adversarial_robust_model.pth`, encapsule 25.6 millions de paramètres dans un fichier de 282 MB. Il a été entraîné pendant 16 epochs avant déclenchement de l’early stopping, avec un pic de performance observé à l’époque 14. Le temps total d’entraînement de 96 heures sur CPU reflète le coût computationnel élevé mais nécessaire de l’adversarial training. Les métriques finales confirment le succès de l’approche : 100% de clean accuracy, 98.7% d’adversarial accuracy pendant l’entraînement, et 100% d’accuracy sur l’ensemble de validation. Le robustness gap de 1.3% est remarquablement faible, témoignant d’une convergence quasi-parfaite entre performance nominale et robustesse adversariale.

Un système de vérification automatique valide trois aspects critiques de l’implémentation, assurant la conformité aux spécifications. Il confirme d’abord l’existence et la taille du dataset augmenté, vérifiant que plus de 1000 images ont effectivement été générées dans les répertoires appropriés. Il teste ensuite la capacité du modèle à générer des exemples adversariaux PGD conformes, s’assurant que la perturbation maximale reste dans les limites du budget epsilon (0.031 plus une tolérance numérique de 10^{-6}). Enfin, il contrôle que le robustness gap sauvegardé dans le checkpoint final est bien inférieur au seuil de 5% fixé comme objectif de certification. Ces vérifications automatisées garantissent que l’implémentation fonctionne conformément aux spécifications et que les résultats observés ne sont pas dus à des erreurs de configuration ou de calcul.

4.2.6 Synthèse et perspectives

Ce chapitre a présenté l'implémentation complète d'un système de sécurisation adversariale intégrant trois axes de défense complémentaires : augmentation du dataset (multiplication par 15.2), diversification à quatre niveaux de transformations ciblant spécifiquement la robustesse adversariale, et entraînement adversarial avec génération PGD dynamique. L'architecture modulaire centralise ces mécanismes dans un pipeline cohérent qui a permis d'atteindre des performances exceptionnelles : 100% de clean accuracy, 98.7% d'adversarial accuracy, et un robustness gap de seulement 1.3%.

L'observation du zero robustness gap à l'époque 14 valide empiriquement l'efficacité de l'approche et remet en question la perception du trade-off robustesse/accuracy comme inévitable. Le modèle a appris des représentations qui ne sacrifient pas la performance nominale pour obtenir la robustesse, mais optimisent simultanément les deux objectifs. Cette convergence suggère que, sur des tâches bien définies avec un dataset de qualité, il est possible de dépasser le dilemme traditionnel documenté dans la littérature sur des benchmarks complexes comme ImageNet.

Le chapitre suivant (Section 4.4) présentera l'évaluation exhaustive de ce modèle sur l'ensemble de test, confronté à une batterie d'attaques comprenant FGSM, PGD standard, PGD renforcé, et AutoAttack, permettant une comparaison quantitative rigoureuse avec le baseline vulnérable et une validation de la robustesse réelle (par opposition à une robustesse artificielle due à du gradient masking).

Bibliographie

- [1] Zhang, H., Yu, Y., Jiao, J., Xing, E. P., Ghaoui, L. E., & Jordan, M. I. (2019). *Theoretically Principled Trade-off between Robustness and Accuracy*. International Conference on Machine Learning (ICML), 7472-7482.
- [2] Hendrycks, D., & Dietterich, T. (2019). *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*. International Conference on Learning Representations (ICLR).
- [3] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). *Towards Deep Learning Models Resistant to Adversarial Attacks*. International Conference on Learning Representations (ICLR).
- [4] Schmidt, L., Santurkar, S., Tsipras, D., Talwar, K., & Madry, A. (2018). *Adversarially Robust Generalization Requires More Data*. Advances in Neural Information Processing Systems (NeurIPS), 31.

Conclusion Générale

Bibliographie

- [1] APDP - bénin.
- [2] Introduction à l'apprentissage automatique - CultureSciences de l'ingénieur - éducol STI. Consulté à l'adresse https://sti.educol.education.fr/si-ens-paris-saclay/ressources_pedagogiques/introduction-a-lapprentissage-automatique.
- [3] ISO/IEC 42001 :2023.
- [4] Proposition de RÈGLEMENT DU PARLEMENT EUROPÉEN ET DU CONSEIL ÉTABLISSANT DES RÈGLES HARMONISÉES CONCERNANT L'INTELLIGENCE ARTIFICIELLE (LÉGISLATION SUR L'INTELLIGENCE ARTIFICIELLE) ET MODIFIANT CERTAINS ACTES LÉGISLATIFS DE L'UNION.
- [5] N. Akhtar and A. Mian. Threat of adversarial attacks on deep learning in computer vision : A survey.
- [6] S. Akkour, A. Haounani, and F. Assadi. LA PROTECTION DES DONNÉES PERSONNELLES FACE À L'INTELLIGENCE ARTIFICIELLE. 4(3). Number : 3.
- [7] D. Andre. Qu'est-ce qu'un modèle ?
- [8] P. Besse, A. Besse-Patin, and C. Castets-Renard. Implications juridiques et éthiques des algorithmes d'intelligence artificielle dans le domaine de la santé. (8). Publisher : Société française de statistique (SFdS).
- [9] B. Biggio, B. Nelson, and P. Laskov. Poisoning attacks against support vector machines.
- [10] M. Brundage, S. Avin, J. Clark, H. Toner, P. Eckersley, B. Garfinkel, A. Dafoe, P. Scharre, T. Zeit-zoff, B. Filar, H. Anderson, H. Roff, G. C. Allen, J. Steinhardt, C. Flynn, S. . hÉigearthaigh, S. J. Beard, H. Belfield, S. Farquhar, C. Lyle, R. Crootof, O. Evans, M. Page, J. Bryson, R. Yampolskiy, and D. Amodi. The malicious use of artificial intelligence : Forecasting, prevention, and mitigation.
- [11] J. Buolamwini and T. Gebru. Gender shades : Intersectional accuracy disparities in commercial gender classification. In *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, pages 77–91. PMLR. ISSN : 2640-3498.
- [12] N. Carlini, C. Liu, . Erlingsson, J. Kos, and D. Song. The secret sharer : Evaluating and testing unintended memorization in neural networks.

-
- [13] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun. Dermatologist-level classification of skin cancer with deep neural networks. 542(7639) :115–118. Publisher : Nature Publishing Group.
- [14] S. Fazelpour and D. Danks. Algorithmic bias : Senses, sources, solutions. 16(8) :e12760.
- [15] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples.
- [16] T. Gu, B. Dolan-Gavitt, and S. Garg. BadNets : Identifying vulnerabilities in the machine learning model supply chain.
- [17] K. Hill. The secretive company that might end privacy as we know it.
- [18] J. Hurwitz and D. Kirsch. Machine learning for dummies (ibm limited edition), 2018. Consulté à l'adresse <https://www.ibm.com/downloads/cas/GB8ZMQZ3>.
- [19] Z. C. Lipton. The mythos of model interpretability.
- [20] K. Lum and W. Isaac. To predict and serve? 13(5) :14–19.
- [21] V. B.-H. Mayor, Philip. Wrongfully arrested because face recognition cant tell black people apart | ACLU.
- [22] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan. A survey on bias and fairness in machine learning. 54(6) :115 :1–115 :35.
- [23] D. Oliynyk, R. Mayer, and A. Rauber. I know what you trained last summer : A survey on stealing machine learning models and defences. 55(14) :324 :1–324 :41.
- [24] Z. Pan, W. Yu, X. Yi, A. Khan, F. Yuan, and Y. Zheng. Recent progress on generative adversarial networks (GANs) : A survey. 7 :36322–36333.
- [25] N. Parmar, A. Vaswani, J. Uszkoreit, L. Kaiser, N. Shazeer, A. Ku, and D. Tran. Image transformer. In *Proceedings of the 35th International Conference on Machine Learning*.
- [26] M. T. Ribeiro, S. Singh, and C. Guestrin. "why should i trust you?" : Explaining the predictions of any classifier.
- [27] M. Sharif, S. Bhagavatula, L. Bauer, and M. K. Reiter. Accessorize to a crime : Real and stealthy attacks on state-of-the-art face recognition. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16*, pages 1528–1540. Association for Computing Machinery.
- [28] R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models.
- [29] J. Steinhardt, P. W. Koh, and P. Liang. Certified defenses for data poisoning attacks.
- [30] P. Zaraté. L'intelligence artificielle d'aujourd'hui à aujourd'hui. *Droit Social, Dalloz, C, 2*, 2021.

Table des matières

Dédicace	ii
Remerciements	iii
Résumé	iv
Abstract	v
List of Figures	vi
List of Tables	vii
Liste des Algorithmes	viii
Liste des acronymes	ix
Introduction	1
1 Cadre conceptuel et théorique	3
Introduction	3
1.1 Introduction aux modèles d'intelligence artificielle	3
1.1.1 Notion d'intelligence artificielle	3
1.1.2 Les différents types de modèles d'IA	3
1.1.2.1 Types d'apprentissage en intelligence artificielle	4
1.1.2.2 Architectures avancées des modèles d'IA	5
Importance et impact de l'IA dans les systèmes modernes	6
Sécurité et intelligence artificielle	7
Définitions et concepts clés	7
Les menaces et vulnérabilités liées aux modèles d'IA	8
Illustration de l'évolution des menaces et des défenses	8
Usages des modèles d'IA dans les systèmes de maintien de l'ordre	9
Contexte de l'intégration de l'IA dans la sécurité publique	9
Principaux cas d'usage dans les forces de l'ordre	9

1.3.3	Risques et limites observés	10
1.3.4	Nécessité de régulation et d'encadrement	11
		11
1.4	Éthique et cadre réglementaire des modèles IA	11
1.4.1	Principes fondamentaux de l'éthique en IA	11
1.4.2	Régulations et normes	12
1.5	Typologie des vulnérabilités des modèles d'IA	13
1.5.1	Vulnérabilités structurelles et inhérentes aux modèles IA	13
		13
1.5.2	Vulnérabilités liées aux données d'entraînement	13
1.5.2.1	Empoisonnement des données (Data Poisoning) :	13
1.5.2.2	Biais et discrimination algorithmique :	13
1.5.2.3	Fuites d'informations sensibles :	14
		14
1.5.3	Vulnérabilités liées à l'inférence et à l'exploitation des modèles	15
1.5.3.1	Vulnérabilités par inférence de modèle :	15
1.5.3.2	Vulnérabilités par extraction de modèle	15
1.5.3.3	–	15
		15
1.6	Classification des attaques contre les modèles d'IA	15
1.6.1	Attaques adversariales	16
		16
		16
1.6.2	Attaques sur les données d'entraînement	16
1.6.2.1	Poisoning Attacks (attaques d'empoisonnement)	17
1.6.2.2	Backdoor Attacks (attaques par porte dérobée)	17
		17
1.6.3	Attaques sur la confidentialité et la robustesse des modèles	18
1.6.3.1	Membership Inference Attack (MIA)	18
1.6.3.2	Model Extraction Attacks :	19
		19
		19
1.7	Études de cas et exemples dans le domaine sécuritaire	20
1.7.1	Erreurs d'identification dans la reconnaissance faciale	20
1.7.2	Surveillance biaisée et prédictions injustes	20
1.7.3	Contournement ou manipulation des systèmes IA	20
		20
2	Méthodologie et Architecture	22
	Introduction	22
2.1	Démarche méthodologique	22
2.1.1	Collecte des Données	22
2.1.2	Modélisation d'une Architecture de sécurité	22
2.1.3	Mise en place d'un cas d'expérimental	22
2.1.4	Simulation et Évaluation des Résultats	22

2.2	Méthode de sécurisation des modèles IA dans les systèmes de sécurité	22
2.2.1	Architecture globale de protection des modèles IA	22
2.2.1.1	Principes fondamentaux de l'architecture	22
2.2.1.2	Architecture en cinq zones de sécurité	24
2.2.2	Approche DevSecOps dans la protection des modèles IA.	29
2.2.2.1	Pipeline CI/CD sécurisé	29
	Particularités pour l'IA :	29
	Structure du pipeline en cinq étapes :	29
		30
		30
2.2.2.2	Containérisation avec Docker	32
	Configuration sécurisée pour modèles IA :	33
2.2.2.3	Monitoring et alerting en production	33
2.2.2.4	Recommandations éthiques des modèles d'IA .	34
	Conclusion	35
3	Expérimentation	36
	Introduction	36
3.1	Présentation du Cas expérimental et choix du modèle.	36
3.1.1	Objectif de l'expérience	36
		37
3.1.2	Scénario d'application : Détection d'objets dangereux	37
3.1.2.1	Contexte opérationnel	37
3.1.2.2	Classification binaire	38
3.1.3	Justification du choix de ResNet50	38
3.1.3.1	Architecture et capacité	38
3.1.3.2	Vulnérabilité adversariale documentée	38
3.1.3.3	Adaptabilité au scénario	39
3.2	Évaluation des vulnérabilités de ResNet50	39
3.3	Implémentation du Pipeline d'entraînement complet	39
3.4	Pipeline de déploiement en production	39
	Conclusion	39
4	Résultats et discussion	40
4.1	Évaluation des vulnérabilités de ResNet50	40
4.1.1	Constitution du dataset expérimental	40
4.1.1.1	Architecture du chargement des données	40
4.1.1.2	Évolution du dataset : de la limitation initiale à l'augmentation	41
4.1.2	Métriques d'évaluation de la robustesse	41
4.1.2.1	Métriques de performance classiques	41
4.1.2.2	Métriques de robustesse adversariale	42
4.1.3	Protocole d'attaque et évaluation baseline	42
4.1.3.1	Entraînement du modèle baseline	42
4.1.3.2	Implémentation des attaques adversariales	42
4.1.3.3	Pipeline d'évaluation complet	43

4.1.4	Résultats de vulnérabilité du baseline	43
4.1.4.1	Performance empirique obtenue	43
4.1.4.2	Analyse des vulnérabilités identifiées	44
4.1.4.3	Certification de sécurité baseline	44
Bibliographie		46
4.2	Implémentation de la méthode de sécurisation	46
4.2.1	Architecture globale du système sécurisé	46
4.2.2	Augmentation et diversification du dataset	47
4.2.3	Entraînement adversarial avec PGD	48
4.2.4	Mécanismes de régulation et d'optimisation	49
4.2.5	Convergence et performance du modèle final	50
4.2.6	Synthèse et perspectives	51
Bibliographie		52
Conclusion		53
Bibliographie		54
Bibliographie		54
Table des matières		56